

Disaster Risk Monitoring Using Satellite Imagery



DEEP
LEARNING
INSTITUTE



DISASTER RISK MONITORING
USING SATELLITE IMAGERY
SELF-PACED

APPLIED DEEP LEARNING AND SATELLITE IMAGERY SEGMENTATION FOR FLOOD RISK ASSESSMENT

BY

Shubham Sharma

© Copyright 2025 by Shubham Sharma – All Rights Reserved

This document, including all methodologies, analysis, and graphical representations, is the original intellectual property of the author, Shubham Sharma.

ACKNOWLEDGEMENT

The successful compilation of this **Technical Review and Methodology Summary** is a direct result of the high-quality, specialized training provided by leading global organizations.

The author, **Shubham Sharma**, wishes to express sincere gratitude to the following entities:

- **The NVIDIA Deep Learning Institute (DLI):** For developing and delivering the cutting-edge technical curriculum, which provided essential hands-on proficiency in **Deep Learning, Computer Vision, and GPU-accelerated computing**.
- **UNITAR (United Nations Institute for Training and Research) and UNOSAT (Operational Satellite Applications Programme):** For their critical partnership in providing the real-world context and data scenarios, thereby validating the practical utility of this technology in **Disaster Risk Monitoring** and humanitarian applications.

This document serves as a permanent portfolio record of the technical and problem-solving skills gained from this invaluable course.

ABOUT THE AUTHOR

Shubham Sharma is a B.Sc. student majoring in Computer Science and Mathematics at the University of Delhi. He has a strong interest in Artificial Intelligence, Machine Learning, Computer Vision, and Robotics. Shubham enjoys building practical projects that solve real problems and help people, and believes that learning becomes meaningful when ideas are turned into working prototypes.

Despite being early in his academic journey, he has already developed a foundation in AI through hands-on practice and self-driven exploration. His curiosity about how machines can see, understand, and assist humans drives most of his work

Certifications & Learning

Shubham has completed several industry-recognized courses that strengthened his understanding of AI and data:

- **NVIDIA, UNITAR & UNOSAT** — Disaster Risk Monitoring Using Satellite Imagery
- **IBM** — Data Fundamentals
- **NASA** – Open Science 101

These certifications reflect his commitment to building strong fundamentals early in his career.

Projects

BrainDesk-AI

A hands-free voice assistant for desktop, built by Shubham as a self-initiated application. It features voice interaction, an adaptive interface, and batch automation, and more—all created to explore practical AI integration in everyday computing.

Computer Vision Application

A YOLO and OpenCV based application that detects surrounding objects, faces, and basic emotions. This project helped him understand how computer vision models work in real-time scenarios.

Interests

Shubham is enthusiastic about AI, robotics, and computer vision. He enjoys learning by building, experimenting, and steadily developing his technical skills as he moves forward in his engineering journey.

TABLE OF CONTENTS

ACKNOWLEDGEMENT..... ii

About the Author iii

Introduction.....1

Learning Overview & Environment Setup.....3

Chapter 16

Chapter 228

Chapter 352

Chapter 464

References.....76

INTRODUCTION

Purpose of This Document

This document serves as a comprehensive technical summary and portfolio review of the specialized course, **Disaster Risk Monitoring Using Satellite Imagery**. The objective is to move beyond mere certification and provide **verifiable, structured evidence** of competency in applied Deep Learning and Geospatial Analysis.

While the primary source of knowledge is the curriculum developed by the **NVIDIA Deep Learning Institute (DLI)** in collaboration with **UNITAR** and **UNOSAT**, the content herein is authored by **Shubham Sharma**. It acts as a professional record detailing the end-to-end technical methodology, from data preprocessing to model performance evaluation.

The High-Stakes Problem: Automated Flood Detection

The core focus of this technical review is the application of advanced Deep Learning models to solve a time-critical, humanitarian problem: the **rapid assessment of infrastructure damage** due to flood events.

In environments where immediate human access is impossible or unsafe, the ability to utilize high-resolution satellite imagery (specifically **Sentinel-1** data, characterized by multi-spectral bands and GeoTIFF format) is paramount. The project specifically addresses how to use **Computer Vision** techniques to automatically classify damaged structures at a pixel level, providing immediate, actionable intelligence for response teams.

The End-to-End Deep Learning Pipeline

The solution documented in this report follows a complete, industry-standard Deep Learning workflow, leveraging NVIDIA's specialized software stack for maximum efficiency:

1. **Data Ingestion and Preprocessing:** Use of the **NVIDIA DALI (Data Loading Library)** to create GPU-accelerated data pipelines. This approach is critical for high-throughput data loading and performing on-the-fly **Data Augmentation** (such as Random Rotation) directly on the GPU, maximizing computational efficiency.
2. **Model Training:** Utilizing the **NVIDIA TAO (Train Adapt Optimize) Toolkit** for efficient **Transfer Learning**. This process involves fine-tuning a pre-trained **ResNet-18** model adapted for a **U-Net Semantic Segmentation** architecture. This method accelerates convergence and improves accuracy, even with complex satellite imagery datasets.
3. **Deployment and Optimization:** Exporting the trained model and compiling it into a highly optimized **TensorRT** engine. The final model is then deployed via the **NVIDIA**

Triton Inference Server, allowing for real-time inference via network APIs while optimizing for crucial production metrics like **throughput** and **latency**.

Key Technical Competencies Demonstrated

This document serves to validate practical skills in the following high-demand areas:

- **Semantic Segmentation:** Proficiency in a core Computer Vision technique for pixel-level classification.
- **Geospatial Data Handling:** Expertise in working with complex, multi-band satellite data (GeoTIFFs) and their associated annotation challenges.
- **Performance Engineering:** GPU-accelerated tools (**DALI**, **TAO Toolkit**, **TensorRT**) to build an efficient, production-ready inference solution.
- **Model Deployment:** Ability to use enterprise-level inference servers (**Triton**) and understand the trade-offs between latency and throughput.

The following sections will detail the specific methodologies, configuration parameters, and results of each stage of this pipeline.

Learning Overview & Environment Setup

Introduction

This book begins with the core objective of the NVIDIA DLI course: understanding how to build a complete flood-detection system using satellite imagery. The workflow focuses on segmentation of flooded regions using data captured by the **Sentinel-1** satellite constellation.

Learning Objectives

In this course, you will learn important concepts about using satellite imagery to develop deep learning-based disaster risk monitoring systems, including:

- The Application of Computer Vision for Disaster Risk Monitoring
- Manipulation of Data Collected by Earth Observation Satellites
- Ways to Efficiently Process Large Imagery Data
- Deep Learning Model Development Challenges
- End-to-End Machine Learning Workflow

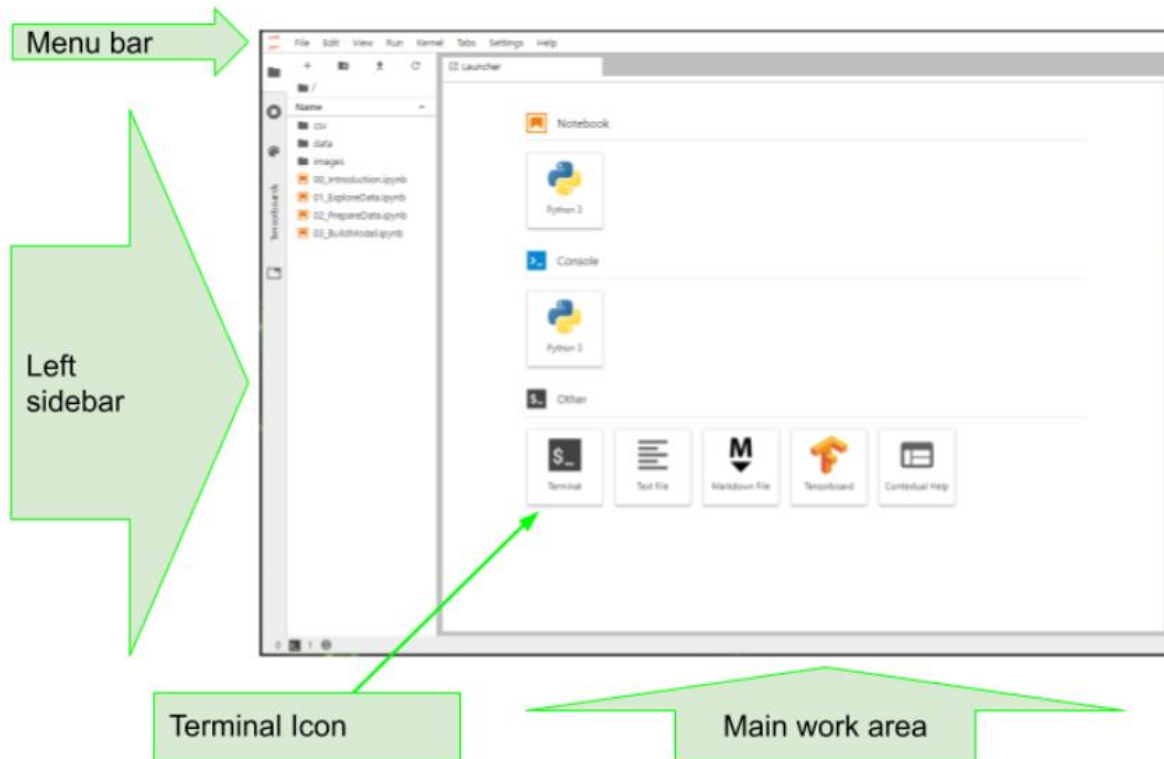
JupyterLab

For this hands-on work, we use **JupyterLab** as the primary development environment. JupyterLab provides an integrated interface where you can write code, view outputs, manage files, and access a Linux terminal—all in one place.

When you first launch JupyterLab, you'll see:

- A **menu bar** at the top
- A **file browser** on the left
- A **main work area**, which initially opens with this introduction notebook

This setup allows you to navigate the project folders, run Python cells interactively, and control the entire workflow from a single interface.



Environment Setup & Practice

Try executing the simple print statement in the cell below.

```
# DO NOT CHANGE THIS CELL
# activate this cell by selecting it with the mouse or arrow keys then use the keyboard shortcut [Shift+Enter] to execute
print('This is just a simple print statement')
```

Now try executing the terminal command in the cell below.

```
# DO NOT CHANGE THIS CELL
# activate this cell by selecting it with the mouse or arrow keys then use the keyboard shortcut [Shift+Enter] to execute
!echo 'This is another simple print statement'
```

NVIDIA GPU Cloud

Before proceeding, you must have an **NVIDIA NGC (NVIDIA GPU Cloud) account** along with a valid **API key**. These are essential for accessing the pre-trained models, containers, and other resources used throughout the workflow.

If you have not set this up already, make sure to:

- **Create and activate a free NGC account**

- **Generate your NGC API key** and store it securely for later use

Replace `<MY_NGC_API_KEY>` with your key and execute the cell to set the variable.

```
apikey='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'

# DO NOT CHANGE THIS CELL
# write the config file
config_dict={'apikey': apikey, 'format_type': 'json', 'org': 'nvidia'}
with open('config', 'w') as f:
    f.write(';WARNING - This is a machine generated file. Do not edit manually.\n')
    f.write(';WARNING - To update local config settings, see "ngc config set -h"\n')
    f.write('\n[CURRENT]\n')
    for k, v in config_dict.items():
        f.write(k+'='+v+'\n')

# preview the config file
!cat config
```

```
;WARNING - This is a machine generated file. Do not edit manually.
;WARNING - To update local config settings, see "ngc config set -h"
```

```
[CURRENT]
apikey=[your_api_key]
format_type=json
org=nvidia
```

```
# DO NOT CHANGE THIS CELL
# move the config file to ~/.ngc
!mkdir -p ~/.ngc & mv config ~/.ngc/
```

```
# DO NOT CHANGE THIS CELL
# Login to NGC's docker registry
!docker login -u '$oauthtoken' -p $apikey nvcr.io
```

```
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store
```

Login Succeeded

```
# DO NOT CHANGE THIS CELL
# pull TAO docker container images
!docker pull nvcr.io/nvidia/tao/tao-toolkit:5.0.0-tf1.15.5
!docker pull nvcr.io/nvidia/tao/tao-toolkit:5.5.0-deploy
```

```
b5ef13ee: Pulling fs layer
3ff61f07: Pulling fs layer
ac7d66df: Pulling fs layer
9e1da3b4: Pulling fs layer
6b6d858e: Pulling fs layer
29520245: Pulling fs layer
6bb9c19e: Pulling fs layer
2f1c7ea0: Pulling fs layer
517d19bd: Pulling fs layer
cc81db56: Pulling fs layer
2fd753fa: Pulling fs layer
36af415f: Pulling fs layer
4d4ebf9c: Pulling fs layer
a1cb86b: Extracting 2.432GB/2.432GBB
```

Chapter 1: Disaster Risk Monitoring Systems and Data Pre-Processing

Introduction

In this foundational chapter, we establish the critical motivation behind disaster risk monitoring and introduce the powerful, hardware-accelerated tools required to handle the massive scales of satellite image data essential for this task.

1. Disaster Risk Monitoring

Natural disasters—including **flood, wildfire, drought, and severe storms**—represent one of the greatest global challenges. These catastrophes wreak havoc throughout the world, resulting in billions of dollars in damages, the destruction of critical infrastructure, and the uprooting of communities, ecosystems, and local economies.

The ability to accurately **detect, quantify, and potentially forecast** these natural phenomena is no longer just an academic pursuit; it is a critical necessity for minimizing their adverse impacts on both human lives and financial stability.

While the primary technical focus of this study is on developing a robust **flood detection** system—a clear and tangible target for building an AI pipeline—it is essential to note that the underlying methods and core framework are highly adaptable. The ultimate goal is to create similar applications that can be quickly deployed for monitoring *any* major natural disaster globally.

2. Flood Detection

A **flood** is fundamentally defined as an overflow of water that submerges land which is typically dry. These events are not monolithic; they can manifest under several distinct conditions, often simultaneously:

1. **Riverine Overflow:** The most familiar type, occurring when an excessive volume of water in a river, stream, or other water body overtops or breaks through its natural or man-made boundaries, such as levees.
2. **Areal Flooding:** The accumulation of rainwater on saturated ground, where the rate of precipitation exceeds the ground's ability to absorb it or the local drainage system's capacity to carry it away.
3. **Channel Exceedance:** When the total flow rate of a water course surpasses the maximum capacity of its designated river channel, forcing water out into the surrounding landscape.

The Climate Change Reality:

Unfortunately, the frequency, intensity, and geographical spread of major flooding events are demonstrably on the rise, primarily driven by **climate change** and the resulting **sea level rise**. This escalation has rightfully garnered significant international attention in recent years. Organizations like the United Nations now place effective response and proactive risk assessment for floods high on the agenda of their **Sustainable Development Goals (SDGs)**.

An Interdisciplinary Data Challenge

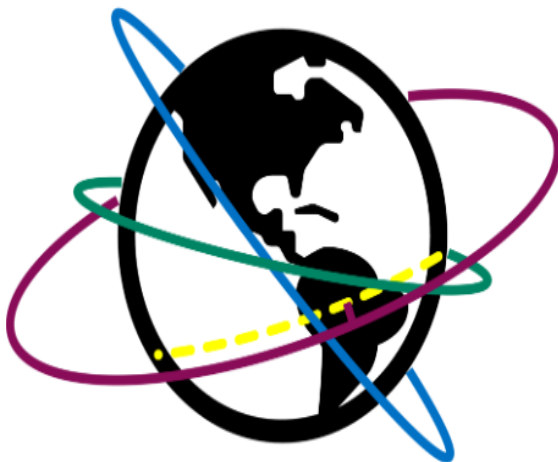
The research into flood events and their evolution is a deeply interdisciplinary study. To build a comprehensive detection and forecasting system, scientists and data practitioners require data from a variety of heterogeneous sources:

- **Earth Observation Data:** Live satellite imagery and surface reflectance data provide the immediate visual evidence of inundation.
- **Hydrological Inputs:** Metrics such as precipitation levels, runoff rates, soil moisture content, snow cover, and snow water equivalent are crucial for modeling the water cycle.
- **Environmental Context:** Detailed topography (terrain elevation) and meteorology (weather patterns) provide the necessary context to understand *why* the water flowed where it did.

The ability to instantly **detect a flood** and accurately **measure the extent of the disaster** using this data is paramount. It allows decision-makers to develop swift, tactical responses for relief operations and, just as importantly, enables scientists to study flood behavior over long periods. This scientific foundation is what ultimately informs the long-term mitigation strategies required to achieve greater global sustainability.

3. Satellite Imagery: The Eye in the Sky

To effectively monitor and mitigate flood events, we turn to **Satellite Imagery**. This remote sensing technique offers decisive advantages over ground-based methods. Physical access to flooded regions is often severely limited or outright dangerous, making manual or human-in-the-loop solutions inefficient and risky. Satellites, by contrast, offer a safe, scalable, and highly efficient solution for capturing the necessary data across vast areas.



Orbit Type	Description
Low Earth Orbit LEO (<1,000 km)	Almost all human activity in space is in LEO due to desirable speed of orbit and data transfer
Medium Earth Orbit MEO (1,000 km - 35,786 km)	MEO comprises of orbits between LEO and GEO. This is commonly used by navigation satellites but is also used by a variety of satellites with different applications
Geosynchronous Earth Orbit GEO (35,786 km)	Satellites in GEO circle the Earth above the equator following Earth's rotation, which make them appear "stationary" over a fixed position. This is commonly used by communication satellites
Elliptical (Not Pictured)	Least common of satellite orbits

Currently, thousands of human-made satellites orbit the Earth. These are placed in various orbits—often dictated by their specialized mission. While some are dedicated to communication or navigation, many, including the ones we utilize in this study, are deployed for **Earth observation** to help scientists study our planet's surface dynamics.

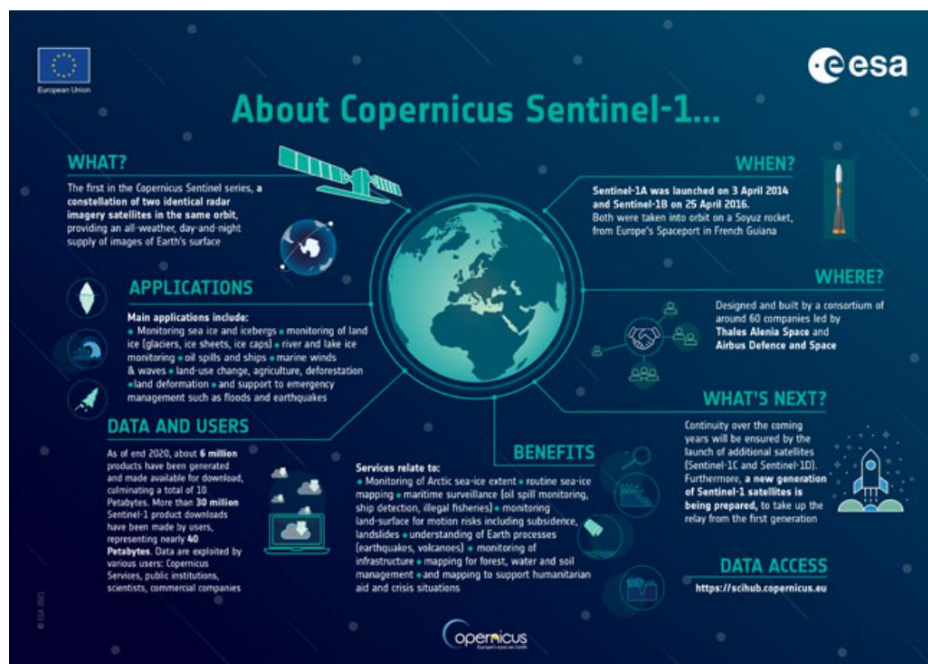
Leveraging Synthetic Aperture Radar (SAR)

Different Earth observation satellites are equipped with unique capabilities tailored to their purpose. For detailed and uninterrupted flood monitoring, the [Copernicus Sentinel-1](#) mission is paramount. This mission provides C-band [Synthetic Aperture Radar \(SAR\)](#) data.

SAR technology holds a decisive advantage over conventional optical satellites (which rely on visible or near-infrared light) because it can operate **day and night** and, crucially, **under cloud cover**. This capability is non-negotiable for disaster response, as floods often occur during or immediately after severe weather. SAR works by emitting microwave pulses and measuring the backscattered signal to create two-dimensional images or three-dimensional reconstructions of the landscape.

The Sentinel-1 mission is comprised of two polar-orbiting satellites, **Sentinel-1A and Sentinel-1B**. Operating in the [Lower Earth Orbit \(LEO\)](#), these satellites maintain a rapid **repeat cycle of just 6 days**. LEO satellites offer the benefits of faster orbital speed and data transfer, making the Sentinel-1 mission uniquely suited for continuous, near-real-time monitoring of flood risk over time.

Ultimately, deploying a **real-time AI-based remote flood level estimation system via Sentinel-1 data** proves to be a game-changing capability for humanitarian and governmental response organizations.



More information about the Sentinel-1 mission can be found [here](#).

4. Computer Vision: Deep Learning for Imagery

At the heart of this type of disaster risk monitoring system is one or more **machine learning models** designed to generate insights from input data. These are generally deep learning neural network models that have been trained for a specific computer vision task. There are numerous approaches for drawing insight from images using machine learning:

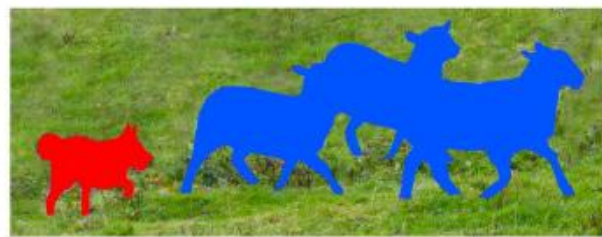
- **Classification:** This is the simplest task, used for identifying the primary object or concept in an entire image. The output is a single label for the entire frame (e.g., "Flood Image"). This is too general for risk assessment as it provides no location or boundary information.
- **Object Detection and Localization:** This approach specifies the location and class of multiple objects in a frame by drawing **bounding boxes** around them (**Localization**). While it provides the general area of a flood, it lacks the pixel-level accuracy required for precise area calculation and resource allocation.
- **Segmentation:** This is the most powerful technique for mapping and measuring environmental features, as it provides **pixel-level accuracy** by creating a fine-grained mask around the detected object. This capability is **essential** for precise flood area measurement. Applications range from AI-powered green screens to autonomous driving.

Segmentation is further divided into two primary types:

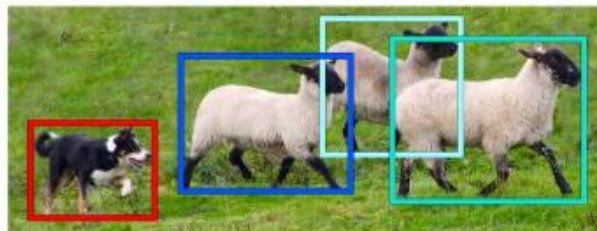
1. **Semantic Segmentation:** Associates every pixel of an image with a class label, such as "flood" or "not-flood". Critically, it treats multiple objects of the same class (e.g., disconnected flood pools) as a single overall entity.
- 2.
3. **Instance Segmentation:** Treats multiple objects of the same class as **distinct, individual instances**. This is typically used for counting objects (e.g., counting individual people or cars).



Image Recognition



Semantic Segmentation



Object Detection



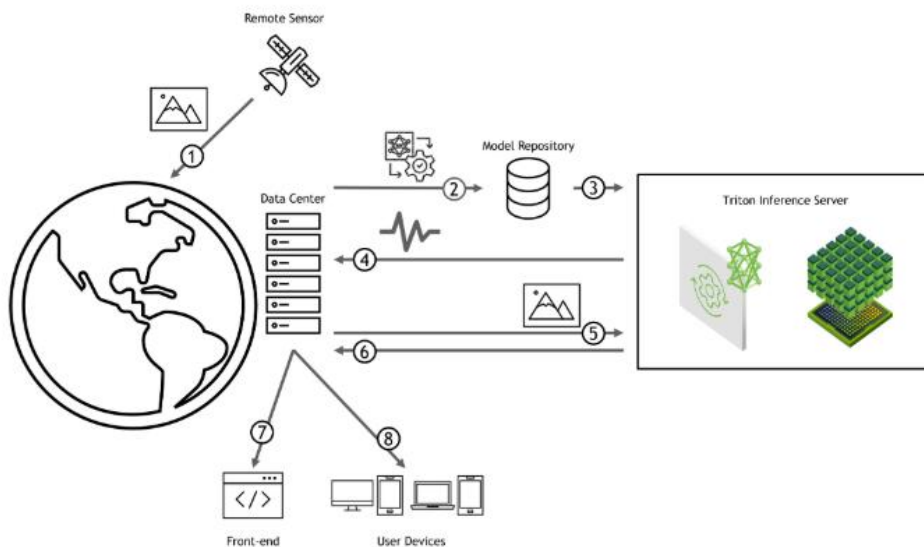
Instance Segmentation

For the specific and critical purpose of detecting and accurately measuring the spatial extent of flood events, we will develop a **Semantic Segmentation model**. This model will be trained with labelled images generated from the Sentinel-1 SAR data to create a high-fidelity, pixel-perfect flood map.

5. Deep Learning-Based Disaster Risk Monitoring System

The system that we envision consists of the below workflow:

1. Satellite remote sensors capture data.
2. Data are used to (continuously) train deep learning neural network models.
3. Different models and versions are managed by the model repository.
4. Model inference performance is actively monitored.
5. Data are passed to the inference server.
6. The deep learning inference results are post-processed for either:
 - Further analytics by a 3rd party or
 - Raising alerts.

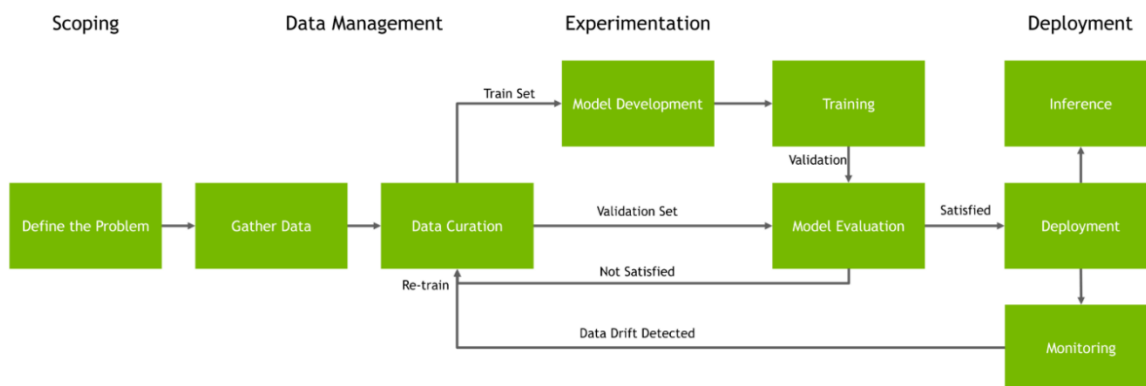


When processing data in real-time, this system can help us in **delineating open water flood areas**. In addition, identifying flood levels will enable effective disaster response and mitigation. If we combine the flood extent mapping with other data such as topography and/or population, we

can create a **plan of action** with downstream applications. We can also use this information to **predict the direction of flow of water, redirect flood waters, organize resources for distribution**, etc. Importantly, such a system can recommend a **path of least flood levels** in real-time that disaster response professionals can potentially adopt.

6. Deep Learning Model Training Workflow

Building a robust deep learning model is a systematic process encompassing several critical stages: **collecting large, high-quality datasets, preparing the data, training the model**, and **optimizing the final model** for deployment.



At its core, training a neural network model using **supervised learning** leverages the network's capacity to perform automatic feature extraction from raw data, linking those features to our desired target output (the flood mask).

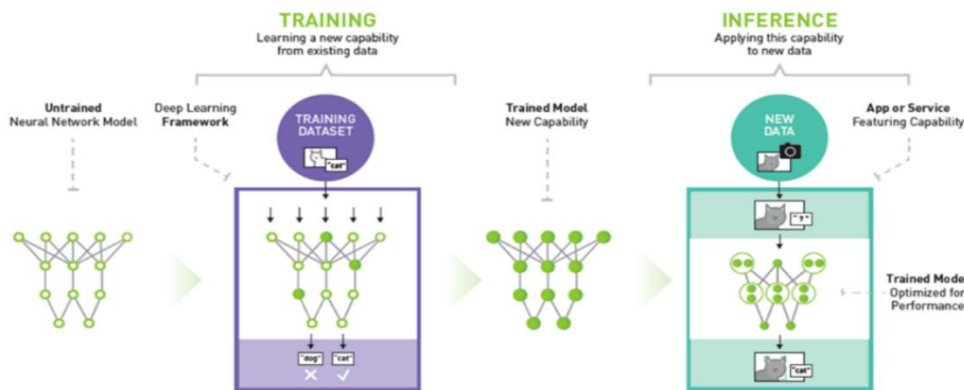
A crucial step in this workflow is splitting the original dataset into **training** and **validation/test sets**. This partitioning is essential to prevent **overfitting**, a condition where the model performs exceptionally well on the data it was trained on but fails to generalize accurately to new, unseen data. While deep learning model performance generally increases when trained with more data, this process is inherently time-consuming and computationally intensive.

Once the model is trained and properly evaluated, it moves to deployment, where it can be used for **inference** (making predictions). Even post-deployment, the model can be further fine-tuned and optimized to deliver the right balance of **accuracy** and **performance**.

Ultimately, the goal of any best-in-class deep learning solution is to holistically consider several key deployment factors:

- **Model Accuracy** (the precision of the prediction).
- **Inference Throughput** (the volume of predictions per second).
- **Latency** (the time taken for a single prediction).

- **Computational Cost** (the required hardware resources).



7. Deep Learning Challenges

Developing and deploying deep learning-based solutions presents several common, complex challenges:

- **Data Scarcity and Cost:** Training accurate deep learning models from scratch requires a large amount of data, and acquiring and annotating this data (often manually) is a costly and time-consuming process.
- **Framework Complexity:** Development often requires specialized knowledge of one or more deep learning frameworks, such as [TensorFlow](#), [PyTorch](#), or [Caffe](#).
- **Optimization Effort:** Models require significant effort to fine-tune before they are truly optimized for **inference** and considered production-ready.
- **Real-Time Performance:** Processing high-volume data in real-time is computationally intensive and needs to be facilitated by specialized software and hardware that enables **low latency** and **high throughput**.

As we will demonstrate throughout this report, NVIDIA's specialized software stack—including the [DALI](#) (Data Loading Library), [TAO Toolkit](#) (Train Adapt Optimize), [TensorRT](#), and [Triton Inference Server](#)—can be used to effectively tackle these challenges, creating a streamlined, high-performance pipeline.

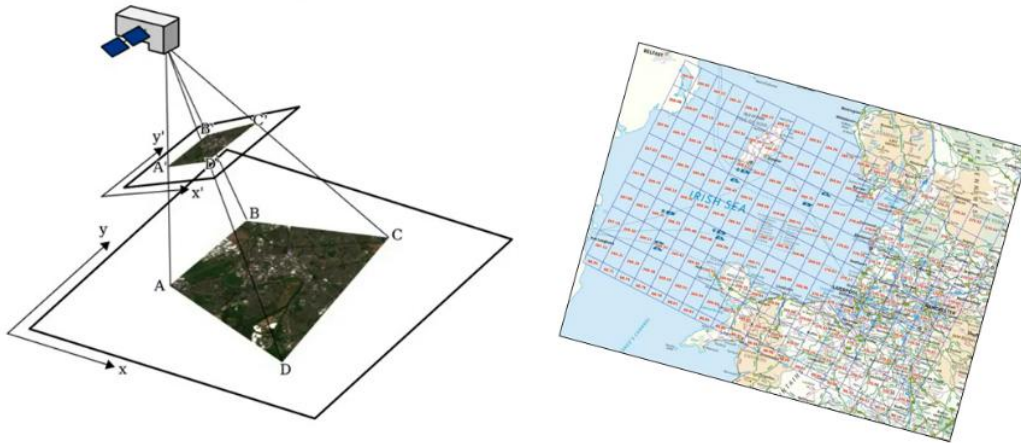
A crucial step in this workflow is splitting the original dataset into **training** and **validation/test sets**. This partitioning is essential to prevent **overfitting**, a condition where the model performs exceptionally well on the data it was trained on but fails to generalize accurately to new, unseen data. While deep learning model performance generally increases when trained with more data, this process is inherently time-consuming and computationally intensive.

Once the model is trained and properly evaluated, it moves to deployment, where it can be used for **inference** (making predictions). Even post-deployment, the model can be further fine-tuned and optimized to deliver the right balance of **accuracy** and **performance**.

Introducing the Dataset

1. Sentinel-1 Data Public Access

The Sentinel-1 SAR data we will use is available from [ESA](#) via the [Copernicus Open Access Hub](#). They maintain an archive and is committed to delivering data within 24 hours of acquisition and maintains recent months of data. They are also available via NASA's [EARTHDATASEARCH](#) or [Vertex](#), Alaska Satellite Facility's data portal. They are organized as tiles, which is the process of subdividing geographic data into pre-defined roughly-squares. Tile-based mapping efficiently renders, stores, and retrieves image data.



2. Data Annotation

Acquiring the necessary data for any high-quality deep learning project is inherently a costly process, and it often represents the single largest bottleneck in the development workflow. To build our semantic segmentation model, we require precise **labelled masks at the pixel level** for each of the Sentinel-1 images, distinguishing flooded areas from non-flooded areas. These annotations are crucial for supervised learning.

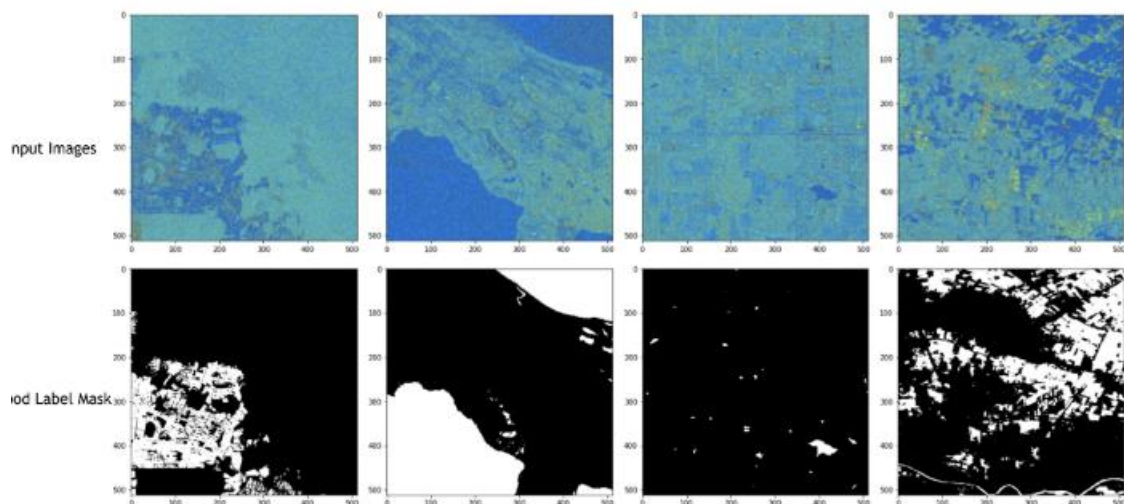
The high-quality data used in this work has been generously provided by [Cloud to Street](#) [1]. More information about the comprehensive dataset can be found [here](#).

[1] Bonafilia, D., Tellman, B., Anderson, T., Issenberg, E. 2020. Sen1Floods11: a georeferenced data set to train and test deep learning flood algorithms for Sentinel-1. The IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops, 2020, pp. 210–211.

3. Exploratory Data Analysis

We have organized the `flood_data` directory for easy processing. For each image tile, there are three pieces of information: the Sentinel-1 `image`, the corresponding labelled `mask`, and a `catalog.json` file. The `catalog.json` describes the metadata associated with the image, such as its geographic coordinates boundaries (longitude, latitude) and timestamp. In addition, there is a top-level `Sen1Floods11_Metadata.geojson` file that records the overall manifest, organizing the data by specific region at a specific time.

```
root@server:/flood_data$ tree
.
├── catalog
│   └── sen1floods11_hand_labeled_source
│       ├── region_1
│       │   └── region_1.json
│       ├── region_2
│       │   └── region_2.json
├── images
│   └── all_images
│       ├── region_1.png
│       └── region_2.png
├── masks
│   └── all_masks
│       ├── region_1.png
│       └── region_2.png
└── Sen1Floods11_Metadata.geojson
```



First, we need to unzip the `flood_data.zip` file, which includes all of the data. We can count the number of images and masks to ensure they match.

```
# Unzip data file
```

```
!unzip data/flood_data.zip -d flood_data
```

```
# import dependencies
```

```
import os
import json
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import matplotlib.gridspec as gridspec
from mpl_toolkits.basemap import Basemap
```

```
# Set environment variables
```

```
%set_env LOCAL_DATA_DIR=/dli/task/flood_data
```

```
# Set paths for images and masks
```

```
image_dir=os.path.join(os.getenv('LOCAL_DATA_DIR'), 'images')
mask_dir=os.path.join(os.getenv('LOCAL_DATA_DIR'), 'masks')
```

```
# Define function to count number of images per region
```

```
def count_num_images(file_dir):
    """ This function returns a dictionary representing the count of images for each
    reason as the key."""
    # List all files in the directory
    file_list=os.listdir(file_dir)
    region_count={}
    # Iterate through the file_list and count by region
    for file_name in file_list:
        region=file_name.split('_')[0]
        if (len(file_name.split('.'))==2) and (region in region_count):
            region_count[region]+=1
        elif len(file_name.split('.'))==2:
            region_count[region]=1
    return region_count
```

```
# Count images and masks by region
```

```
images_count=count_num_images(os.path.join(image_dir, 'all_images'))
masks_count=count_num_images(os.path.join(mask_dir, 'all_masks'))
```

```
# Display Counts
```

```
print(f'-----number of images: {sum(images_count.values())}-----')
display(sorted(images_count.items(), key=lambda x: x[1]))

print(f'-----number of masks: {sum(masks_count.values())}-----')
display(sorted(masks_count.items(), key=lambda x: x[1]))
```

-----number of images: 446-----

```
[('Bolivia', 15),  
 ('Nigeria', 18),  
 ('Somalia', 26),  
 ('Pakistan', 28),  
 ('Mekong', 30),  
 ('Spain', 30),  
 ('Sri-Lanka', 42),  
 ('Ghana', 53),  
 ('Paraguay', 67),  
 ('India', 68),  
 ('USA', 69)]
```

-----number of masks: 446-----

```
[('Bolivia', 15),  
 ('Nigeria', 18),  
 ('Somalia', 26),  
 ('Pakistan', 28),  
 ('Mekong', 30),  
 ('Spain', 30),  
 ('Sri-Lanka', 42),  
 ('Ghana', 53),  
 ('Paraguay', 67),  
 ('India', 68),  
 ('USA', 69)]
```

Exercise 1 – Count Input Data

Let's count the number of images in our training dataset.

Each image and mask are accompanied by a JSON file that documents its latitude and longitude. We can go through the manifest to get a sense of how our data is populated. We plot the map using Basemap [Matplotlib Toolkit](#) to display the coverage of our data.

```
# DO NOT CHANGE THIS CELL
# define function to get coordinates from catalog
def get_coordinates(catalog_dir):
    """
    This function returns a list of boundaries for every image as [[lon, lat], [lon, lat], [lon, lat], etc.] in the catalog.
    """
    catalog_list=os.listdir(catalog_dir)
    all_coordinates=[]
    for catalog in catalog_list:
        # check if it's a directory based on if file_name has an extension
        if len(catalog.split('.'))==1:
            catalog_path=f'{catalog_dir}/{catalog}/{catalog}.json'
            # read catalog
            with open(catalog_path) as f:
                catalog_json=json.load(f)
            # parse out coordinates
            coordinates_list=catalog_json['geometry']['coordinates'][0]
            lon=[coordinates[0] for coordinates in coordinates_list]
            all_coordinates.append(lon)
            lat=[coordinates[1] for coordinates in coordinates_list]
            all_coordinates.append(lat)
    return all_coordinates

# DO NOT CHANGE THIS CELL
# set paths for images catalog
image_catalog_dir=os.path.join(os.getenv('LOCAL_DATA_DIR'), 'catalog', 'sen1floods11_hand_labeled_source')
image_coordinates_list=get_coordinates(image_catalog_dir)

# create figure
plt.figure(figsize=(15, 10))

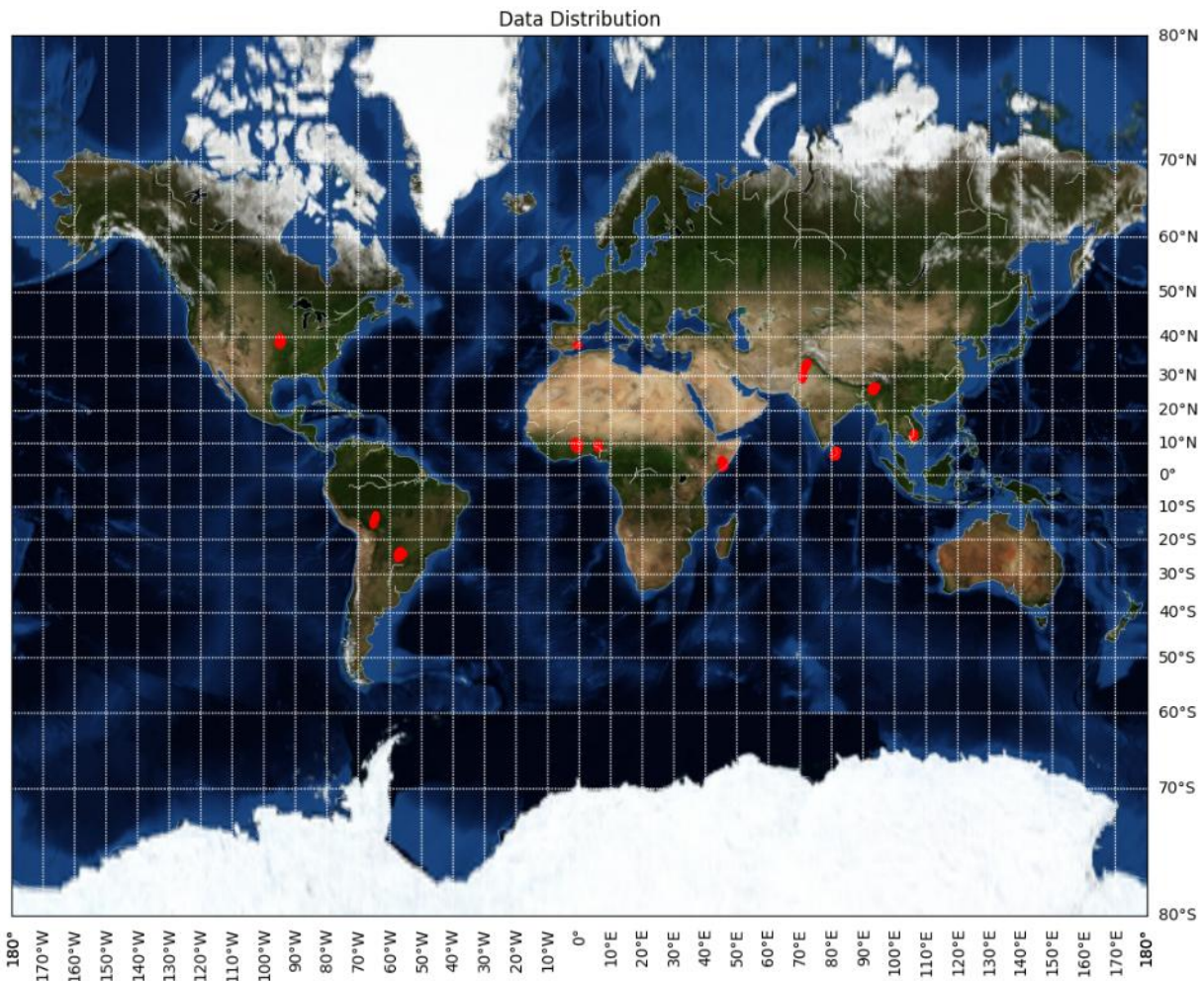
# create a Basemap
m=Basemap(projection='merc', llcrnrlat=-80, urcrnrlat=80, llcrnrlon=-180, urcrnrlon=180)

# display blue marble image
m.bluemarble(scale=0.2) # 0.2 downsamples to 1350x675 image
m.drawcoastlines(color='white', linewidth=0.2) # add coastlines
m.drawparallels(range(-90, 90, 10), labels=[0, 1, 0, 0], color='white', textcolor='black')
m.drawmeridians(range(-180, 180, 10), labels=[0, 0, 0, 1], color='white', textcolor='black', rotation=90)

# flatten lat and lon coordinate lists
image_lon=[image_coordinates_list[x] for x in range(len(image_coordinates_list)) if x%2==0]
image_lon=np.concatenate(image_lon).ravel()
image_lat=[image_coordinates_list[x] for x in range(len(image_coordinates_list)) if x%2==1]
image_lat=np.concatenate(image_lat).ravel()

# convert lon/lat to x/y map projection coordinates
x, y=m(image_lon, image_lat)
plt.scatter(x, y, s=10, marker='o', color='Red')

plt.title('Data Distribution')
plt.show()
```

Out of the entire Earth, we only have a small number of tiles available.

Let's now take look at the specific areas. We have provided three helper functions `get_extent(file_path)`, `tiles_by_region(country_name, plot_type)` and `tiles_by_boundaries(top_left, bottom_right, plot_type)`, where `plot_type` is either `masks` or `images`.

```
# DO NOT CHANGE THIS CELL
# define function to get extent of an image from catalog
def get_extent(file_path):
    """
    This function returns the extent as [left, right, bottom, top] for a given image.
    """
    # read catalog for image
    with open(file_path) as f:
        catalog_json=json.load(f)
    coordinates=catalog_json['geometry']['coordinates'][0]
    coordinates=np.array(coordinates)
    # get boundaries
    left=np.min(coordinates[:, 0])
    right=np.max(coordinates[:, 0])
    bottom=np.min(coordinates[:, 1])
    top=np.max(coordinates[:, 1])
    return left, right, bottom, top
```

```

# DO NOT CHANGE THIS CELL
# define function to plot by region
def tiles_by_region(region_name, plot_type='images'):
    # set catalog and images/masks path
    catalog_dir=os.path.join(os.getenv('LOCAL_DATA_DIR'), 'catalog', 'sen1floods11_hand_labeled_source')
    if plot_type=='images':
        dir=os.path.join(image_dir, 'all_images')
        cmap='viridis'
    elif plot_type=='masks':
        dir=os.path.join(mask_dir, 'all_masks')
        cmap='gray'
    else:
        raise Exception('Bad Plot Type')

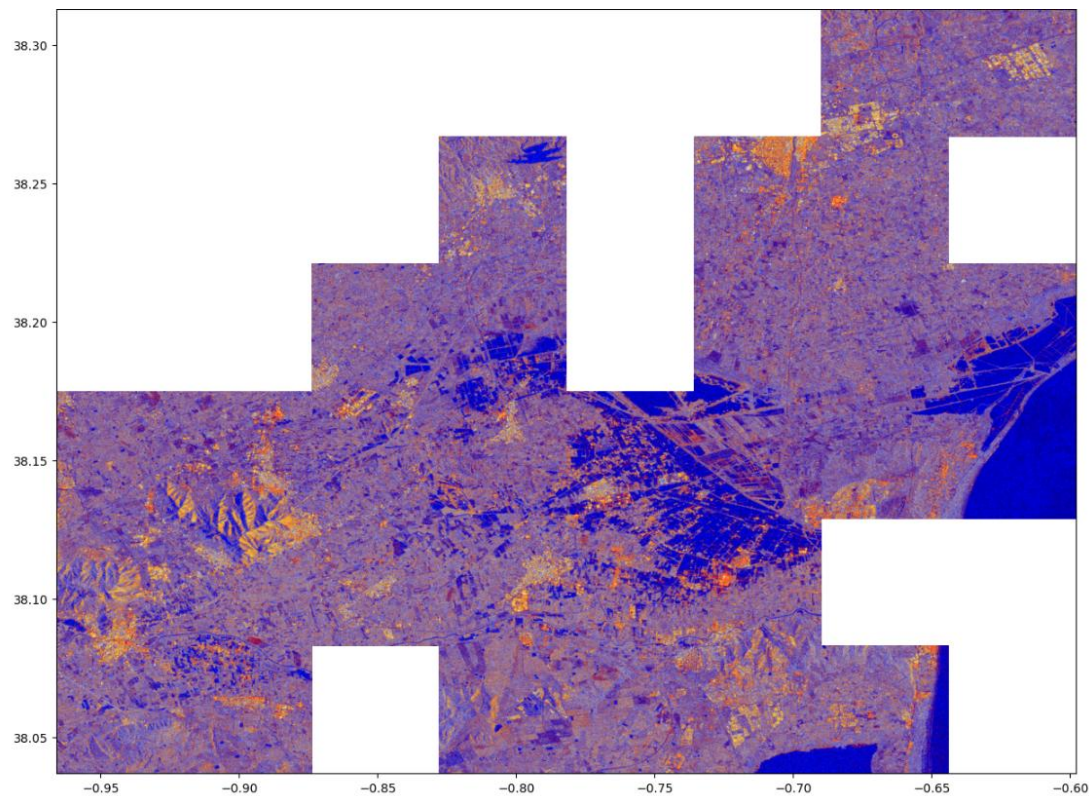
    # initiate figure boundaries, which will be modified based on the extent of the tiles
    x_min, x_max, y_min, y_max=181, -181, 91, -91
    fig=plt.figure(figsize=(15, 15))
    ax=plt.subplot(111)

    # iterate through each image/mask and plot
    file_list=os.listdir(dir)
    for each_file in file_list:
        # check if image/mask is related to region and a .png file
        if (each_file.split('.')[-1]=='png') & (each_file.split('_')[0]==region_name):
            # get boundaries of the image
            extent=get_extent(f"{catalog_dir}/{each_file.split('.')[0]}/{each_file.split('.')[0]}.json")
            x_min, x_max=min(extent[0], x_min), max(extent[1], x_max)
            y_min, y_max=min(extent[2], y_min), max(extent[3], y_max)
            image=mpimg.imread(f'{dir}/{each_file}')
            plt.imshow(image, extent=extent, cmap=cmap)

    # set boundaries of the axis
    ax.set_xlim([x_min, x_max])
    ax.set_ylim([y_min, y_max])
    plt.show()

tiles_by_region(region_name='Spain', plot_type='images')

```




```

# DO NOT CHANGE THIS CELL
# define function to plot by boundaries
def tiles_by_boundaries(left_top, right_bottom, plot_type='images'):
    # set catalog and images/masks path
    catalog_dir=os.path.join(os.getenv('LOCAL_DATA_DIR'), 'catalog', 'sen1floods11_hand_labeled_source')
    if plot_type=='images':
        dir=os.path.join(image_dir, 'all_images')
        cmap='viridis'
    elif plot_type=='masks':
        dir=os.path.join(mask_dir, 'all_masks')
        cmap='gray'
    else:
        raise Exception('Bad Plot Type')

    # initiate figure boundaries, which will be modified based on the extent of the tiles
    x_min, x_max, y_min, y_max=left_top[0], right_bottom[0], right_bottom[1], left_top[1]
    ax_x_min, ax_x_max, ax_y_min, ax_y_max=181, -181, 91, -91

    fig=plt.figure(figsize=(15, 15))
    ax=plt.subplot(111)

    # iterate through each image/mask and plot
    file_list=os.listdir(dir)
    for each_file in file_list:
        # check if image/mask is a .png file
        if each_file.split('.')[-1]=='png':
            # get boundaries of the image/mask
            extent=get_extent(f"{catalog_dir}/{each_file.split('.')[0]}/{each_file.split('.')[0]}.json")
            (left, right, bottom, top)=extent
            if (left>x_min) & (right<x_max) & (bottom>y_min) & (top<y_max):
                ax_x_min, ax_x_max=min(left, ax_x_min), max(right, ax_x_max)
                ax_y_min, ax_y_max=min(bottom, ax_y_min), max(top, ax_y_max)
                image=mpimg.imread(f'{dir}/{each_file}')
                plt.imshow(image, extent=extent, cmap=cmap)

    # set boundaries of the axis
    ax.set_xlim([ax_x_min, ax_x_max])
    ax.set_ylim([ax_y_min, ax_y_max])
    plt.show()

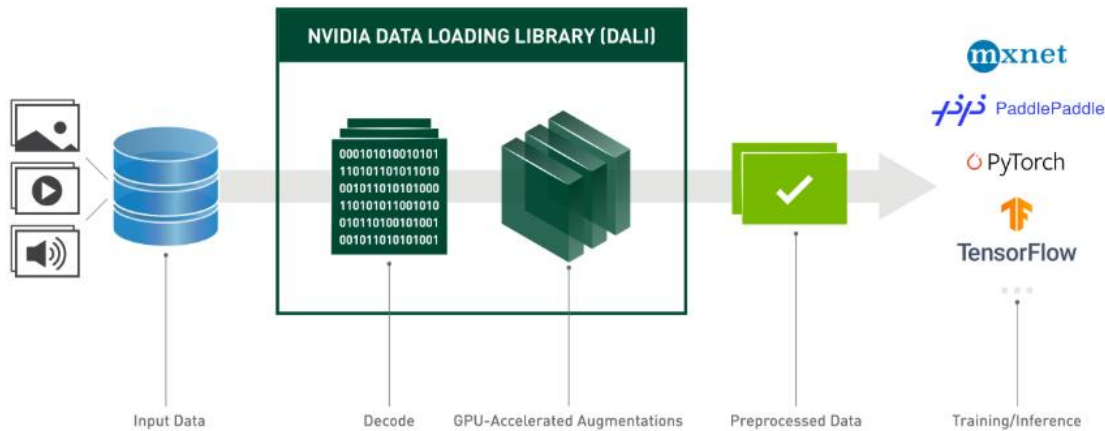
tiles_by_boundaries(left_top=(-0.966, 38.4), right_bottom=(-0.597, 38.0), plot_type='images')

```

4. Data Pre-Processing With DALI

Deep learning models require vast amounts of data to produce accurate predictions, and this need becomes more significant as models grow in size and complexity. Regardless of the model, some degree of **pre-processing** is required for both training and inference. In computer vision applications, this usually includes steps like decoding, resizing, and normalizing the data to a standardized format accepted by the neural network.

Data pre-processing for deep learning workloads has garnered little attention until recently, often eclipsed by the tremendous computational resources required for training complex models. These pre-processing routines, often referred to as pipelines, are currently executed on the **CPU** using libraries such as [OpenCV](#), [Pillow](#), or [Rasterio](#). Today's deep learning applications include complex, multi-stage data processing pipelines consisting of many serial operations. Relying on the CPU to handle these pipelines has become a major **bottleneck** that limits performance and scalability.



The **NVIDIA Data Loading Library (DALI)** is designed specifically to address this issue. DALI is a library for data loading and pre-processing engineered to accelerate deep learning applications. It provides a collection of highly optimized building blocks for loading and processing image, video, and audio data. DALI tackles the problem of the CPU bottleneck by **offloading data pre-processing to the GPU**, drastically improving efficiency.

DALI offers several powerful features:

- It provides data processing primitives for a variety of deep learning applications, supporting most used image file formats.
- It relies on its own execution engine, which is specifically built to maximize the throughput of the input pipeline.
- It can be used as a portable, drop-in replacement for built-in data loaders and data iterators in popular deep learning frameworks.
- Features such as **prefetching, parallel execution, and batch processing** are handled transparently for the user.

Different deep learning frameworks often have multiple, distinct data pre-processing implementations, which results in challenges such as the portability of training and inference workflows and code maintainability. Data processing pipelines implemented using **DALI are portable** because they can easily be retargeted to frameworks like TensorFlow, PyTorch, MXNet, and PaddlePaddle. Furthermore, since the pre-processing routines used for inference are often identical to those used for training, implementing both using DALI can save boilerplate and code repetition across the entire development cycle.

5. DALI Pipeline

At the core of data processing with DALI lies the concept of a data processing **pipeline**. It is composed of multiple operations connected in a directed graph and contained in an object of class `nvidia.dali.Pipeline`. This class provides the ability to define, build, and run data processing pipelines. Each operator in the pipeline typically gets one or more inputs, applies some kind of data processing, and produces one or more outputs. There are special kinds of

operators that don't take any inputs and produce outputs. Those special operators that act like a data source, readers, random number generators and external source fall into this category.

DALI offers CPU and GPU implementations for a wide range of processing operators. The availability of a CPU or GPU implementation depends on the nature of the operator. Make sure to check the documentation for an [up-to-date list of supported operations](#), as it is expanded with every release. The easiest way to define a DALI pipeline is using the `pipeline_def` Python decorator. To create a pipeline, we define a function where we instantiate and connect the desired operators and return the relevant outputs. Then just decorate it with `pipeline_def`.

Let's start with defining a very simple pipeline, which will have two operators. The first operator is a file reader that discovers and loads files contained in a directory. The reader outputs both the contents of the files (in this case, PNGs) and the labels, which are inferred from the directory structure. The second operator is an [image decoder](#). Lastly, we return the image and label pairs. The easiest way to create a pipeline is by using the `pipeline_def` decorator. In the `simple_pipeline` function we define the operations to be performed and the flow of the computation between them. For more information about `pipeline_def` look to the [documentation](#).

```
# import dependencies

from nvidia.dali.pipeline import Pipeline
from nvidia.dali import pipeline_def
import nvidia.dali.fn as fn
import nvidia.dali.types as types

import warnings
warnings.filterwarnings("ignore")
```

```
batch_size = 4

@pipeline_def
def simple_pipeline():
    # use fn.readers.file to read encoded images and labels from the hard drive
    pngs, labels=fn.readers.file(file_root=image_dir)

    # use the fn.decoders.image operation to decode images from png to RGB
    images=fn.decoders.image(pngs, device='cpu')

    # specify which of the intermediate variables should be returned as the output
    return images, labels
```

In order to use the pipeline defined with `simple_pipeline`, we need to create and build it. This is achieved by calling `simple_pipeline()`, which creates an instance of the pipeline. Then we call `build()` on this newly created instance:

```
# create and build pipeline

pipe=simple_pipeline(batch_size=batch_size, num_threads=4, device_id=0)
pipe.build()
```

Notice that decorating a function with `pipeline_def` adds new named arguments to it. They can be used to control various aspects of the pipeline, such as batch size, number of threads used to perform computation on the CPU, and which GPU device to use (though pipeline created with `simple_pipeline` does not yet use GPU for compute). For more information about `Pipeline` arguments you can look to [Pipeline documentation](#)

Once built, a pipeline instance runs in an [asynchronous](#) fashion by calling the pipeline's `run` method to get a batch of results. We unpack the results into `images` and `labels` as expected. Both of these elements contain a list of tensors.

```
# DO NOT CHANGE THIS CELL
# run the pipeline
simple_pipe_output=pipe.run()

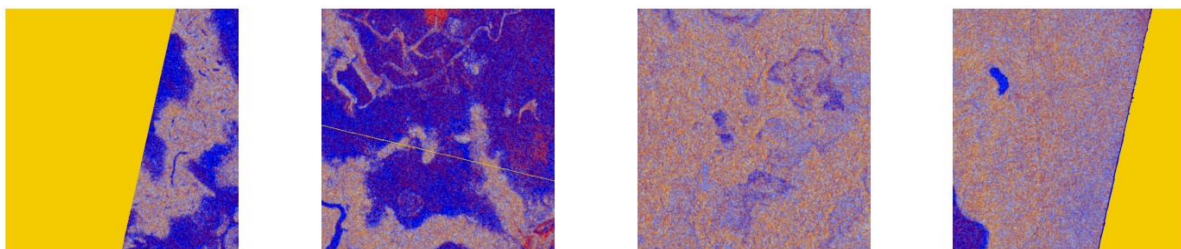
images, labels=simple_pipe_output
print("Images is_dense_tensor: " + str(images.is_dense_tensor()))
print("Labels is_dense_tensor: " + str(labels.is_dense_tensor()))
```

```
Images is_dense_tensor: True
Labels is_dense_tensor: True
```

In order to see the images, we will need to loop over all tensors contained in `TensorList`, accessed with its `at` method.

```
# define a function display images
def show_images(image_batch):
    columns=4
    rows=1
    # create plot
    fig=plt.figure(figsize=(15, (15 // columns) * rows))
    gs=gridspec.GridSpec(rows, columns)
    for idx in range(rows*columns):
        plt.subplot(gs[idx])
        plt.axis("off")
        plt.imshow(image_batch.at(idx))
    plt.tight_layout()

show_images(images)
```



6. Data Augmentation

Deep learning models require training with vast amounts of data to achieve accurate results. DALI can not only read images from disk and batch them into tensors, but it can also perform various **augmentations** on those images to improve deep learning training results.

Data augmentation artificially increases the size of a dataset by introducing random disturbances to the data, such as geometric deformations, color transforms, noise addition, and so on. These disturbances help produce models that are **more robust** in their predictions, **avoid overfitting**, and deliver **better accuracy**. We will use DALI to demonstrate common data augmentation techniques that we will introduce for our model training, such as **cropping**, **resizing**, and **flipping**.

```
# DO NOT CHANGE THIS CELL
import random

@pipeline_def
def augmentation_pipeline():
    # use fn.readers.file to read encoded images and labels from the hard drive
    image_pnxs, _=fn.readers.file(file_root=image_dir)
    # use the fn.decoders.image operation to decode images from png to RGB
    images=fn.decoders.image(image_pnxs, device='cpu')

    # the same augmentation needs to be performed on the associated masks
    mask_pnxs, _=fn.readers.file(file_root=mask_dir)
    masks=fn.decoders.image(mask_pnxs, device='cpu')

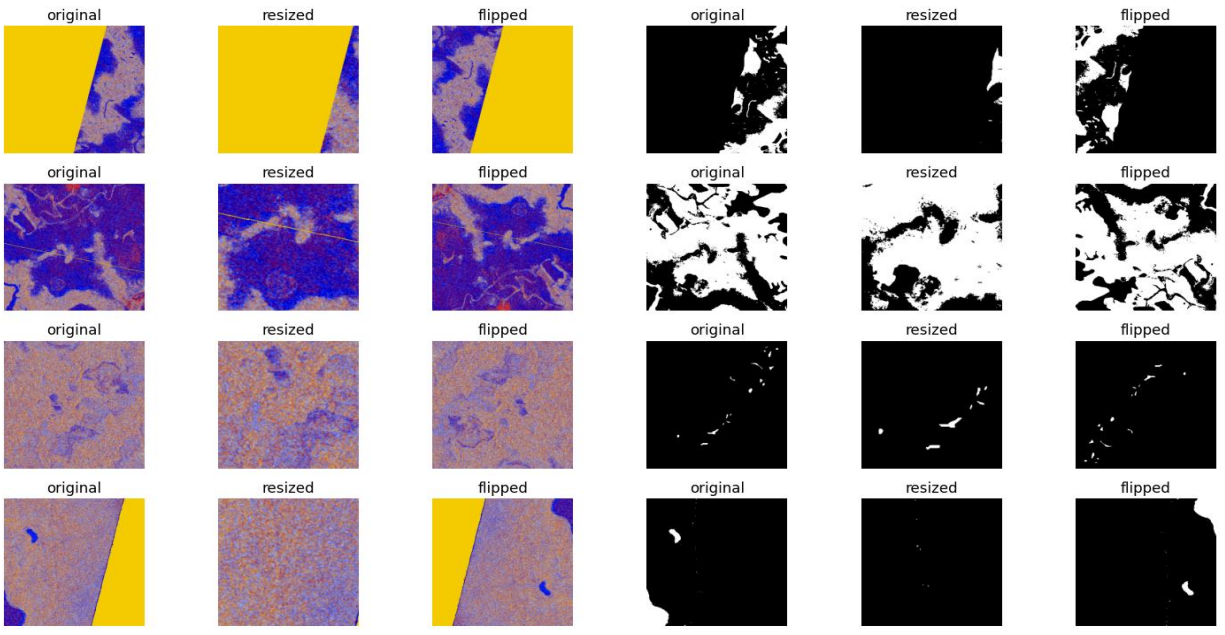
    image_size=512
    roi_size=image_size*.5
    roi_start_x=image_size*random.uniform(0, 0.5)
    roi_start_y=image_size*random.uniform(0, 0.5)

    # use fn.resize to investigate an roi, region of interest
    resized_images=fn.resize(images, size=[512, 512], roi_start=[roi_start_x, roi_start_y], roi_end=[roi_start_x+roi_size, roi_start_y+roi_size])
    resized_masks=fn.resize(masks, size=[512, 512], roi_start=[roi_start_x, roi_start_y], roi_end=[roi_start_x+roi_size, roi_start_y+roi_size])

    # use fn.resize to flip the image
    flipped_images=fn.resize(images, size=[-512, -512])
    flipped_masks=fn.resize(masks, size=[-512, -512])
    return images, resized_images, flipped_images, masks, resized_masks, flipped_masks
```

```
# DO NOT CHANGE THIS CELL
# define a function display images
augmentation=['original', 'resized', 'flipped']
def show_augmented_images(pipe_output):
    image_batch, resized_image_batch, flipped_image_batch, mask_batch, resized_mask_batch, flipped_mask_batch=pipe_output
    columns=6
    rows=batch_size
    # create plot
    fig=plt.figure(figsize=(15, (15 // columns) * rows))
    gs=gridspec.GridSpec(rows, columns)
    grid_data=[image_batch, resized_image_batch, flipped_image_batch, mask_batch, resized_mask_batch, flipped_mask_batch]
    grid=0
    for row_idx in range(rows):
        for col_idx in range(columns):
            plt.subplot(gs[grid])
            plt.axis('off')
            plt.title(augmentation[col_idx%3])
            plt.imshow(grid_data[col_idx].at(row_idx))
            grid+=1
    plt.tight_layout()
```

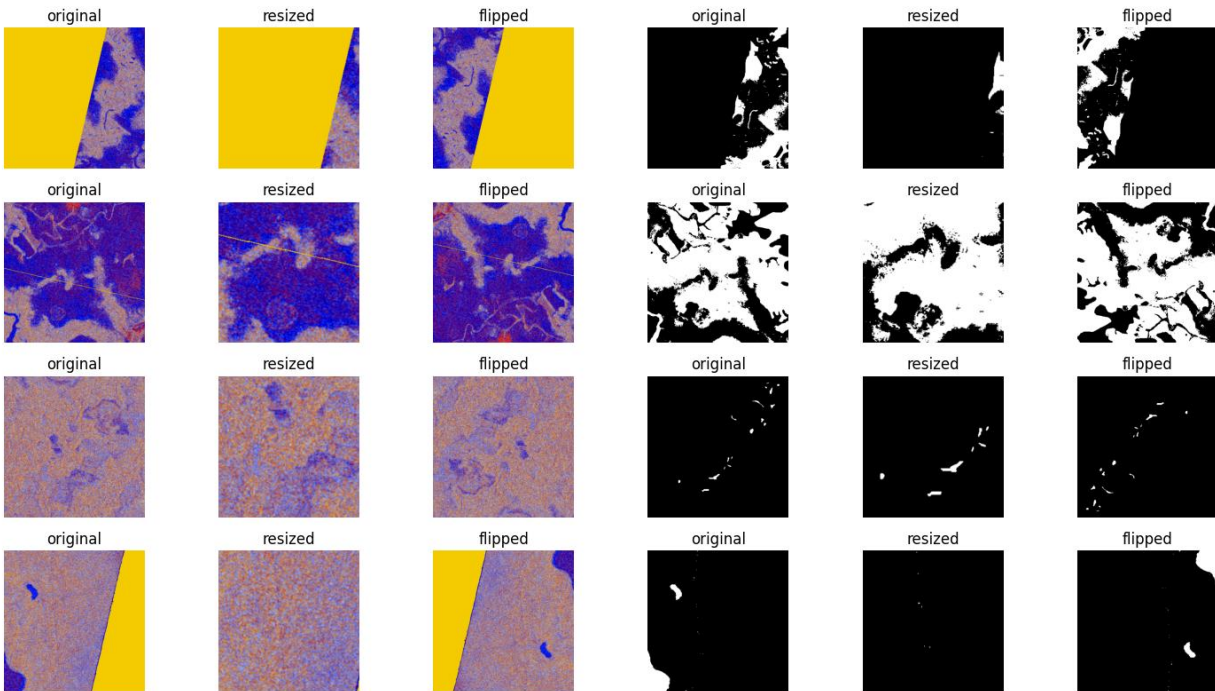
```
# DO NOT CHANGE THIS CELL
show_augmented_images(augmentation_pipe_output)
```



Exercise 2 – Data Augmentation on Batch

Let's perform data augmentation on more batches of data.

```
# DO NOT CHANGE THIS CELL  
show_augmented_images(pipe.run())
```



Random Rotation

Now let us perform additional data augmentation by rotating each image (by a random angle). We can generate a random angle with `random.uniform` and use `rotate` for the rotation. We create another pipeline that uses the GPU to perform augmentations. DALI makes this transition very easy. The only thing that changes is the definition of the rotate operator. We only need to set the device argument to `gpu` and make sure that its input is transferred to the GPU by calling `.gpu()`.


```
# DO NOT CHANGE THIS CELL
@pipeline_def
def rotate_pipeline():
    images, _=fn.readers.file(file_root=image_dir)
    masks, _=fn.readers.file(file_root=mask_dir)
    images=fn.decoders.image(images, device='cpu')
    masks=fn.decoders.image(masks, device='cpu')

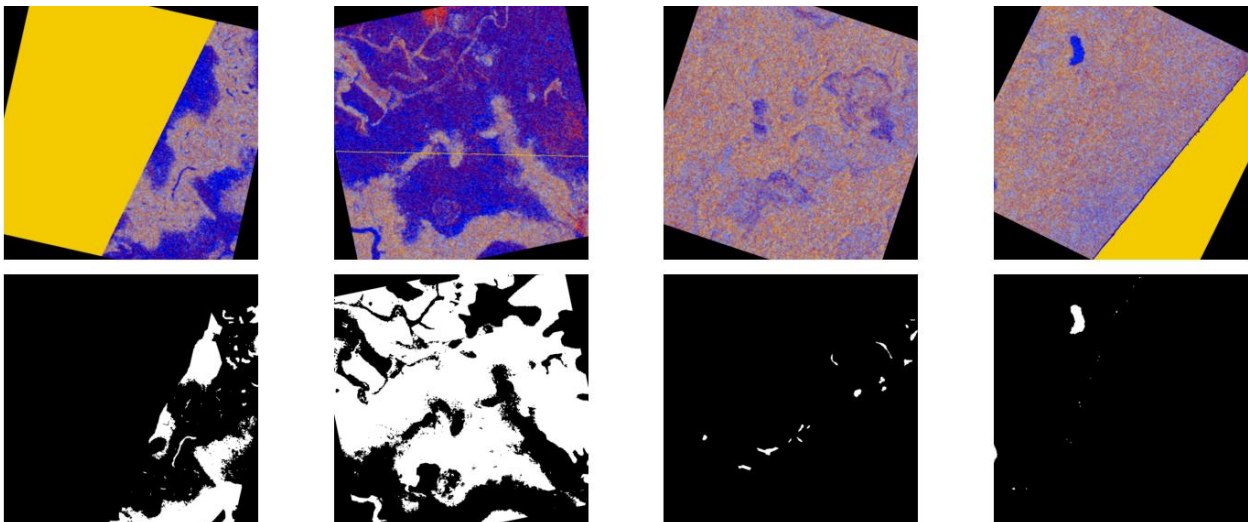
    angle=fn.random.uniform(range=(-30.0, 30.0))
    rotated_images = fn.rotate(images.gpu(), angle=angle, fill_value=0, keep_size=True, device='gpu')
    rotated_masks = fn.rotate(masks.gpu(), angle=angle, fill_value=0, keep_size=True, device='gpu')

    return rotated_images, rotated_masks
```

```
# DO NOT CHANGE THIS CELL
pipe=rotate_pipeline(batch_size=batch_size, num_threads=4, device_id=0)
pipe.build()
rotate_pipe_output=pipe.run()
```

```
def show_rotate_images(pipe_output):
    image_batch, rotated_batch=pipe_output
    columns=batch_size
    rows=2
    fig=plt.figure(figsize=(15, (15 // columns) * rows))
    gs=gridspec.GridSpec(rows, columns)
    grid_data=[image_batch.as_cpu(), rotated_batch.as_cpu()]
    grid=0
    for row_idx in range(rows):
        for col_idx in range(columns):
            plt.subplot(gs[grid])
            plt.axis('off')
            plt.imshow(grid_data[row_idx].at(col_idx))
            grid+=1
    plt.tight_layout()
```

```
# DO NOT CHANGE THIS CELL
show_rotate_images(rotate_pipe_output)
```



Chapter 2: Efficient Model Training

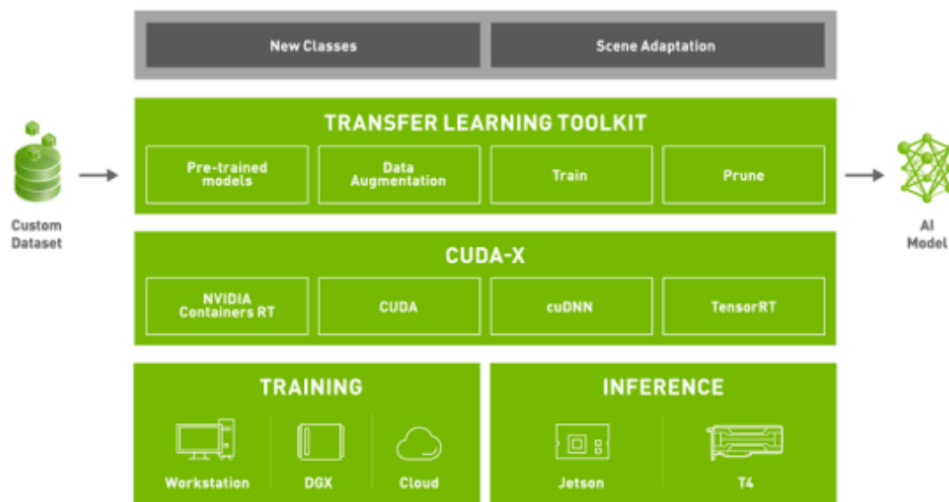
Introduction

In this chapter, we will learn how to train a segmentation model with the **TAO Toolkit** using pre-trained **ResNet-18** weights. In addition, we will learn how to export the model for deployment.

1. Introduction to TAO Toolkit

The **TAO Toolkit (Train Adapt Optimize)** is a comprehensive framework that simplifies the AI/ML model development workflow. It allows developers to fine-tune pre-trained models with custom data to produce highly accurate computer vision models efficiently, eliminating the need for large training runs and deep AI expertise. In addition, it also enables essential **model optimization** for inference performance.

The TAO Toolkit utilizes **transfer learning** with pre-trained models to accelerate the AI development process and significantly reduce the costs associated with large-scale data collection, labelling, and training models from scratch. Transfer learning can be effectively used for classification, object detection, and image segmentation tasks, such as our flood monitoring system.



The key features of the TAO Toolkit include:

- **Low-Coding Approach:** It requires no deep AI framework expertise, reducing the barrier of entry for anyone who wants to get started building AI-based applications.
- **Flexible Configurations:** It allows advanced users to customize and prototype models faster.
- **Large Catalogue of Models:** It provides a vast collection of production-ready pre-trained models for common Computer Vision (CV) tasks that can be customized with a user's own data.

- **Easy Optimization Interface:** It offers an intuitive interface for model optimization techniques such as **pruning** and **quantization-aware training**.
- **Deployment Integration:** It features seamless integration with the **Triton Inference Server** for high-performance deployment.

A key benefit of the TAO Toolkit is that it comes with a set of reference scripts and configuration specifications with default parameter values that enable developers to quickly kick-start training and fine-tuning. This lowers the bar and enables users without a deep understanding of models, expertise in deep learning, or beginning coding skills to be able to train new models and fine-tune the pre-trained ones.

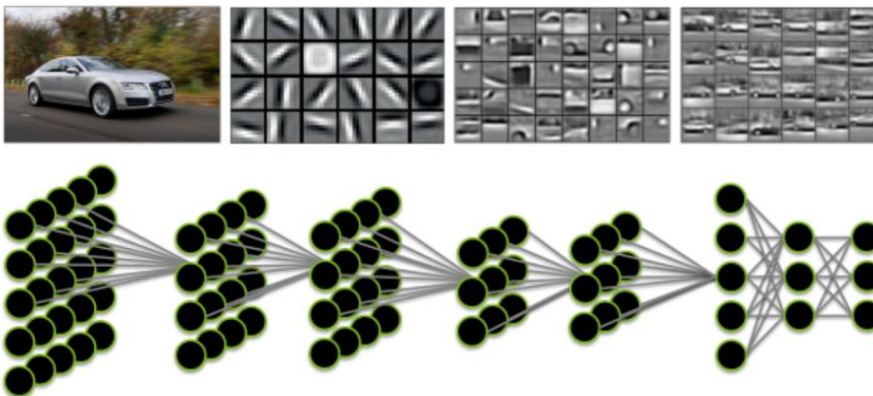
You can learn more about the TAO Toolkit [here](#) or read the documentation [here](#).

2. Transfer Learning

In practice, it is rare and inefficient to initiate the learning task on a network with randomly initialized weights due to factors like data scarcity (inadequate number of training samples) or prolonged training times. One of the most common techniques to overcome this is to use **transfer learning**.

Transfer learning is the process of transferring learned features from one application to another. It is a commonly used training technique where developers use a model trained on one task and re-train to use it on a different task. This works surprisingly well as many of the early layers in a neural network are the same for similar tasks. For example, many of the early layers in a convolutional neural network used for a CV model are primarily used to identify outlines, curves, and other features in an image.

The network formed by these layers are referred to as the **backbone** of a more complex model. Also known as **feature extractors**, they take as input the image and extract the feature map upon which the rest of the network is based. The learned features from these layers can be applied to similar tasks carrying out the same identification in other domains. Transfer learning enables adaptation (fine-tuning) of an existing neural network to a new one, which requires significantly less domain-specific data. In most cases, fine-tuning takes significantly less time (a reduction by x10 factor is common), saving time and resources. As it relates to Vision AI, transfer learning can be used for adding new classification by transferring weights from one application to another



More information about Transfer Learning can be found in this [blog](#).

3. Vision AI Pre-trained Models Supported

Developers, system builders, and software partners building disaster risk monitoring systems can bring their own custom data to train with and fine-tune **pre-trained models** quickly instead of going through significant effort in large data collection and training from scratch. General-purpose vision models provide pre-trained weights for popular network architectures to train an image classification model, an object detection model, or a segmentation model.

This gives users the flexibility and control to build AI models for any number of applications, from smaller lightweight models for edge deployment to larger models for more complex tasks. These models are typically trained on vast datasets like the [Open Images](#) dataset and provide a much better starting point for training versus training from scratch or starting from random weights.

	Image Classification	Object Detection								Segmentation		
		DetectNet_V2	FasterRCNN	SSD	YOLOV3	YOLOV4	RetinaNet	DSSD	EfficientDet	YOLOV4 Tiny	MaskRCNN	UNET
ResNet10/18/34/50/101	✓	✓	✓	✓	✓	✓	✓	✓			✓	✓
VGG16/19	✓	✓	✓	✓	✓	✓	✓	✓				✓
GoogLeNet	✓	✓	✓	✓	✓	✓	✓	✓				
MobileNet V1/V2	✓	✓	✓	✓	✓	✓	✓	✓				
SqueezeNet	✓	✓		✓	✓	✓	✓	✓				
DarkNet 19/53	✓	✓	✓	✓	✓	✓	✓	✓				
CSPDarkNet 19/53	✓					✓				✓		
EfficientNet B0-B5	✓		✓	✓			✓	✓	✓			

The TAO Toolkit adapts popular network architectures and backbones to custom data, allowing developers to train, fine-tune, prune, and export highly optimized and accurate AI models. When working with TAO, first choose the model architecture to be built, then choose one of the supported backbones.

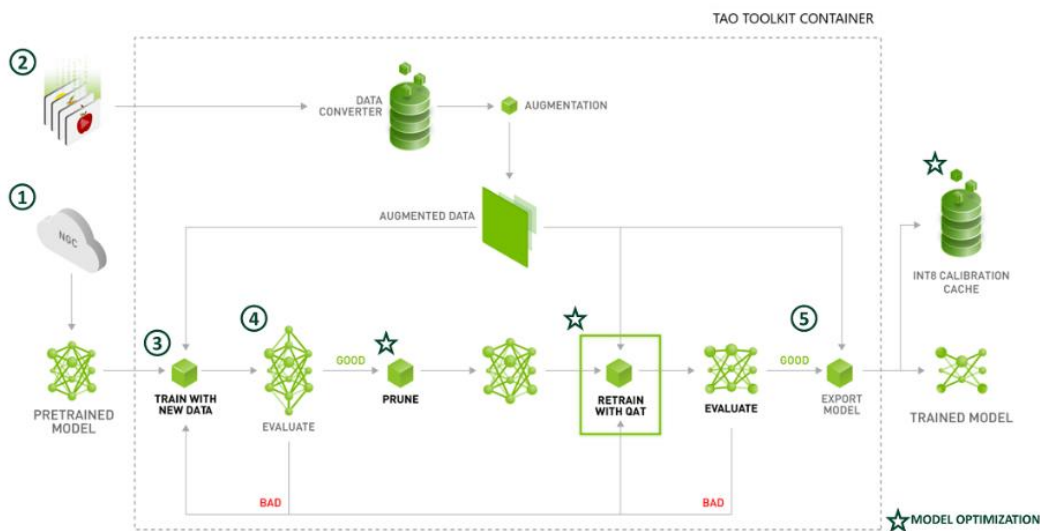
Note: The pre-trained weights from each feature extraction network merely act as a starting point and may not be used without re-training. In addition, the pre-trained weights are network specific and shouldn't be shared across models that use different architectures.

4. TAO Toolkit Workflow

Building disaster risk monitoring systems is difficult, and tailoring even a single component to the needs of the enterprise for deployment is even harder. Deployment for a domain-specific application typically requires several cycles of re-training, fine-tuning, and deploying the model until it satisfies the requirements.

As a starting point, the TAO Toolkit training process typically follows these core steps:

1. **Configuration:** Define all parameters for the training run, including data paths, model architecture, and hyper-parameters, within the Experiment Specification (Spec) files.
2. **Download Pre-Trained Model:** Retrieve a pre-trained model checkpoint from NGC (NVIDIA GPU Cloud) to leverage transfer learning.
3. **Prepare the Data:** Organize the custom dataset (images and labels/masks) into the required directory structure.
4. **Train the Model:** Initiate the training job using transfer learning to fine-tune the pre-trained weights on the custom data.
5. **Evaluate the Model:** Assess the model's performance against the validation data using target metrics (like mIoU for segmentation).
6. **Export the Model:** Convert the trained model checkpoint into an optimized, unencrypted format (like ONNX) suitable for deployment.
7. **Optimize the Model:** Apply further optimization steps, such as pruning or quantization, to improve inference performance (throughput and latency).



5. TAO Launcher, CLI (Command Line Interface), and Spec Files

The TAO Toolkit is designed as a **low-coding framework** to streamline the deep learning workflow. This is achieved through a **launcher** and a powerful **Command Line Interface (CLI)**, eliminating the need for users to write and maintain complex training scripts.

The **TAO Launcher** is the entry point for all operations. It is a Python package that pulls the necessary, highly optimized, and pre-configured deep learning containers from the **NGC registry** (NVIDIA GPU Cloud). Once pulled, it instantly instantiates the appropriate container to perform the desired subtask, such as data conversion, training, evaluation, or export. The first time a task is run, it may take extra time to download the necessary container images.

Users interact with this system using the CLI, which is configured via simple [Protocol Buffer Specification \(Spec\) Files](#). These text files contain all the parameters required for a given experiment, including:

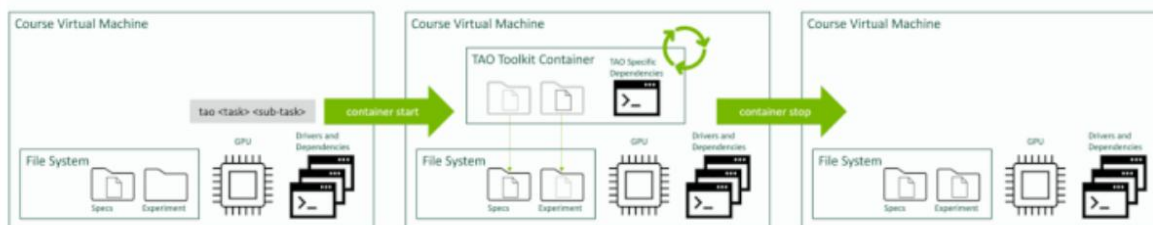
- **Dataset parameters** (e.g., data paths, image size).
- **Model parameters** (e.g., backbone choice, number of classes).
- **Optimizer and training hyperparameters** (e.g., learning rate, epochs).

The TAO CLI convention for invoking tasks is: `tao <model/task> <subtask> <arguments>`

The tasks are grouped into different **task groups**:

- `model`
- `dataset`
- `deploy`

Once the command is executed, the launcher pulls the container, and the specified subtask runs inside the container using the appropriate hardware resources. This containerized approach ensures consistency and optimal performance across different environments.



More information about the TAO Toolkit Launcher can be found in the [TAO Docs](#).

6. Set Up Environment Variables

Before executing any TAO command, it's necessary to set up specific **environment variables**. These variables act as bridges, mapping the local file system paths (where the user stores data and configuration) to the corresponding mount points inside the TAO Docker containers. This ensures that the containers can correctly access the input data and save the outputs (like trained models and logs) back to the desired local directory.

We typically set paths for three crucial variables:

- `$LOCAL_TRAINING_DATA`: The local path to the prepared dataset (images and labels).
- `$LOCAL_SPEC_DIR`: The local path containing the Protocol Buffer Specification (Spec) files used to configure the TAO experiment.
- `$LOCAL_PROJECT_DIR`: The local path designated for storing the output of the TAO experiment. This path is critical because it will house all generated collaterals, such as model checkpoints, logs, and the final exported models (`.tlt` or `.etlt`).

By configuring these variables, we can make sure the TAO experiment collaterals—such as checkpoints, model files (e.g., `.t1t` or `.et1t`), and logs—are output to `$LOCAL_PROJECT_DIR/unet` for our segmentation project.

Encryption Key for IP Protection

*Note that users will be able to define their own **export encryption key** when training from a general-purpose model. This is done to **protect proprietary Intellectual Property (IP)**. The key is used to encrypt the model weights, resulting in an encrypted model file (e.g., `.et1t`). This same key is required later to decrypt and load the `.et1t` model during the deployment phase.*

```
# DO NOT CHANGE THIS CELL
# set environment variables
import os
import json

%set_env KEY=my_model_key

%set_env LOCAL_PROJECT_DIR=/dli/task/tao_project
%set_env LOCAL_DATA_DIR=/dli/task/flood_data
%set_env LOCAL_SPECS_DIR=/dli/task/tao_project/spec_files
os.environ["LOCAL_EXPERIMENT_DIR"]=os.path.join(os.getenv("LOCAL_PROJECT_DIR"), "unet")

%set_env TAO_PROJECT_DIR=/workspace/tao-experiments
%set_env TAO_DATA_DIR=/workspace/tao-experiments/data
%set_env TAO_SPECS_DIR=/workspace/tao-experiments/spec_files
%set_env TAO_EXPERIMENT_DIR=/workspace/tao-experiments/unet

!mkdir $LOCAL_EXPERIMENT_DIR
```

The cell below maps the project directory on your local host to a workspace directory in the TAO docker instance, so that the data and the results are mapped from in and out of the docker. This is done by creating a `.tao_mounts.json` file. For more information, please refer to the [launcher instance](#) in the user guide. Setting the `DockerOptions` ensures that you don't have permission issues when writing data into folders created by the TAO docker.

```
# DO NOT CHANGE THIS CELL
# mapping up the local directories to the TAO docker
mounts_file = os.path.expanduser("~/tao_mounts.json")

drive_map = {
    "Mounts": [
        # Mapping the data directory
        {
            "source": os.environ["LOCAL_PROJECT_DIR"],
            "destination": "/workspace/tao-experiments"
        },
        # Mapping the specs directory.
        {
            "source": os.environ["LOCAL_SPECS_DIR"],
            "destination": os.environ["TAO_SPECS_DIR"]
        },
        # Mapping the data directory.
        {
            "source": os.environ["LOCAL_DATA_DIR"],
            "destination": os.environ["TAO_DATA_DIR"]
        },
    ],
    "DockerOptions": {
        "user": "{}:{}".format(os.getuid(), os.getgid())
    }
}

# writing the mounts file
with open(mounts_file, "w") as mfile:
    json.dump(drive_map, mfile, indent=4)
```


To see the usage of different functionality that are supported, use the `-h` or `--help` option. For more information, see the [TAO Toolkit Quick Start Guide](#). Here is the sample output:

```
# DO NOT CHANGE THIS CELL
```

```
!tao model --help
```

```
tao model unet train [-h] -e <EXPERIMENT_SPEC_FILE>
                        -r <RESULTS_DIR>
                        -n <MODEL_NAME>
                        -m <PRETRAINED_MODEL_FILE>
                        -k <key>
                        [-v Set Verbosity of the logger]
                        [--gpus GPUS]
                        [--gpu_index GPU_INDEX]
```

With the TAO Toolkit, users can train models for object detection, classification, segmentation, optical character recognition, facial landmark estimation, gaze estimation, and more. In TAO's terminology, these would be the tasks, which support subtasks such as `train`, `prune`, `evaluate`, `export`, etc. Each task/subtask requires different combinations of configuration files to accommodate for different parameters, such as the dataset parameters, model parameters, and optimizer and training hyperparameters. Part of what makes TAO Toolkit so easy to use is that most of those parameters are hidden away in the form of experiment specification files (spec files). They are detailed in the [documentation](#) for reference. It's very helpful to have these resources handy when working with the TAO Toolkit. In addition, there are several specific tasks that help with handling the launched commands.

Below are the tasks available in the TAO Toolkit, organized by their respective computer vision objectives. We grayed out the tasks for Conversational AI as they are out of scope for this course.

Task	Classification	Object Detection	Segmentation	Purpose-Built Models	Conversational AI
•list •stop •augment •tlt-converter •download_spec •Info	•classification	•detectnet_v2 •dssd •faster_rcnn •retinanet •ssd •yolo_v3 •yolo_v4	•mask_rcnn •unet	•emotionnet •fpenet •gazenet •gesturenet •heartratenet •lprnet	•intent_slot_classification •punctuation_and_capitalization •question_answering •speech_to_text •text_classification •token_classification

7. U-Net Semantic Segmentation Model

[U-Net](#) is a neural network for image segmentation. This is the type of task we want to perform for our disaster risk monitoring system in order to label each pixel as either `flood` or `notflood`. With the TAO Toolkit, we can choose the desired ResNet-18 backbone as a feature extractor. As such, we will use the `unet` task, which supports the following subtasks:

- train
- evaluate
- inference
- prune
- export

These subtasks can be invoked using the convention `tao model unet <subtask> <args_per_subtask>` on the command-line, where `args_per_subtask` are the arguments required for a given subtask. Additionally, we can always find more information about these subtasks with `tao model unet <subtask> -h` or `tao model unet <subtask> --help`.

8. Preparation for Model Training

For the remainder of the guide, we will use the **TAO Toolkit** to train a semantic segmentation model. The model development workflow involves a few key steps: we start by preparing a pre-trained model and the data, then prepare the configuration file(s), and finally begin to train the model with the new data and evaluate its performance. We will **export** the model once its performance is satisfactory. Note that this guide does not include the advanced **inference optimization** steps, such as pruning and quantization, which are often critical for disaster risk monitoring systems deployed on resource-constrained edge devices.



9. Download Pre-Trained Model

Developers typically begin by choosing and downloading a pre-trained model from **NGC**, which contains pre-trained weights of the architecture of their choice. It's difficult to immediately identify which model/architecture will work best for a specific use case, as there is often a tradeoff between **time to train**, **accuracy**, and **inference performance**. It is common practice to compare across multiple models before picking the best candidate.

Here are some pointers that will help choose an appropriate model:

- **Look at the model inputs/outputs** to decide if it will fit your use case.
- **Input format is an important consideration.** For example, some models expect the input to be 0-1 normalized with input channels in RGB order. Other models that use a different input order may require input preprocessing/mean subtraction that might result in suboptimal performance.

We can use the `ngc registry model list` command to get a list of models that are hosted on the NGC model registry. For example, we can use `registry model list nvidia/tao/*` to list

all available models. The `--column` option identifies the columns of interest. More information about the NGC Registry CLI can be found in the [User Guide](#).

The `ngc registry model download-version <model>/<version>` command will download the model from the registry. It has a `--dest` option to specify the path to the download directory. Alternatively, a catalog of support models can also be found in the [TAO documentation](#).

```
# DO NOT CHANGE THIS CELL
# list all available models
!ngc registry model list nvidia/tao/pretrained_semantic_segmentation:*
```

```
{ "error": "Error: Invalid apikey" }
```

Did you get the below error message? This is likely due to a bad NGC CLI configuration. Reconfigure the CLI using a valid API key from the NGC website.

```
# DO NOT CHANGE THIS CELL
# create directory to store the pre-trained model
!mkdir -p $LOCAL_EXPERIMENT_DIR/pretrained_resnet18/

# download the pre-trained segmentation model from NGC
!ngc registry model download-version nvidia/tao/pretrained_semantic_segmentation:resnet18 \
  --dest $LOCAL_EXPERIMENT_DIR/pretrained_resnet18
```

We will use ResNet18 as the architecture for the semantic segmentation model. [Residual neural network](#), or **ResNet**, is a type of convolutional neural network used as a backbone for many computer vision tasks. The **18** refers to the number of layers in this architecture. It should be noted that typically the deeper (i.e. more layers) a neural network is, the more time consuming it is to compute.

We designated the model to be downloaded locally to `tao_project/unet/pretrained_resnet18`, which is mapped to `/workspace/tao_experiments/unet/pretrained_resnet18` in the TAO container based on the mapping of `LOCAL_PROJECT_DIR` to `TAO_PROJECT_DIR`. Looking at the `local_path` and `transfer_id` keys of the output JSON, we can gather that the path of the pre-trained model should be in the `tao_project/unet/pretrained_resnet18/pretrained_semantic_segmentation_vresnet18` directory. When referencing paths for the TAO Toolkit, it's important to use paths based on the TAO container. In this case it would be `/workspace/tao_experiments/unet/pretrained_resnet18/pretrained_semantic_segmentation_vresnet18`.

```
# DO NOT CHANGE THIS CELL
!tree -a tao_project/unet/pretrained_resnet18
```

```
tao_project/unet/pretrained_resnet18
├─ pretrained_semantic_segmentation_vresnet18
│   └─ resnet_18.hdf5
```

1 directory, 1 file

10. Prepare Dataset

The TAO Toolkit expects the training data for the `unet` subtasks to be in the format described in the documentation. Each mask image is a single-channel image, where every pixel is assigned an integer value that represents the segmentation class `label_id`, as per the mapping provided in the `dataset_config`. Additionally, each image and label must have the same file ID before the extension and size. The image-to-label correspondence is maintained using this filename.

```
/Dataset_01
  /images
    /train
      0000.png
      0001.png
      ...
      N.png
    /val
      0000.png
      0001.png
      ...
      N.png
    /test
      0000.png
      0001.png
      ...
      N.png
  /masks
    /train
      0000.png
      0001.png
      ...
      N.png
    /val
      0000.png
      0001.png
      ...
      N.png
```

The `test` folder in the above directory structure is optional; any folder can be used for inference. Below we will split the data into a train set and a validation set and copy the images into their respective folders.

Below we will split the data into train set and validation set and copy the images into their respective folder.

```
# DO NOT CHANGE THIS CELL
# remove existing splits
!rm -rf $LOCAL_DATA_DIR/images/train
!mkdir -p $LOCAL_DATA_DIR/images/train
!rm -rf $LOCAL_DATA_DIR/images/val
!mkdir -p $LOCAL_DATA_DIR/images/val

!rm -rf $LOCAL_DATA_DIR/masks/train
!mkdir -p $LOCAL_DATA_DIR/masks/train
!rm -rf $LOCAL_DATA_DIR/masks/val
!mkdir -p $LOCAL_DATA_DIR/masks/val
```

```
# DO NOT CHANGE THIS CELL
# import dependencies
from random import sample
import shutil

# define split ratio
split=0.75

# get all images
file_list=os.listdir(f"{os.environ['LOCAL_DATA_DIR']}/images/all_images")
image_count=len(file_list)
train_image_list=sample(file_list, int(image_count*split))
val_image_list=[file for file in file_list if file not in train_image_list]

# move all training images to train directory
for each_file in train_image_list:
    if each_file.split('.')[-1]!='png':
        shutil.copyfile(f"{os.environ['LOCAL_DATA_DIR']}/images/all_images/{each_file}", f"{os.environ['LOCAL_DATA_DIR']}/images/train/{each_file}")
        shutil.copyfile(f"{os.environ['LOCAL_DATA_DIR']}/masks/all_masks/{each_file}", f"{os.environ['LOCAL_DATA_DIR']}/masks/train/{each_file}")

# move all validation images to val directory
for each_file in val_image_list:
    if each_file.split('.')[-1]!='png':
        shutil.copyfile(f"{os.environ['LOCAL_DATA_DIR']}/images/all_images/{each_file}", f"{os.environ['LOCAL_DATA_DIR']}/images/val/{each_file}")
        shutil.copyfile(f"{os.environ['LOCAL_DATA_DIR']}/masks/all_masks/{each_file}", f"{os.environ['LOCAL_DATA_DIR']}/masks/val/{each_file}")
```

11. Model Training

Training configuration is done through a training spec file, which includes options such as which dataset to use for training, which dataset to use for validation, which pre trained model architecture to use, which hyperparameters to tune, and other training options. The `train` , `evaluate` , `prune` , and `inference` subtasks for a U-Net experiment share the same configuration file. Configuration files can be created from scratch or modified using the templates provided in TAO Toolkit's [sample applications](#).

The training configuration file has the following sections:

- `dataset_config`
- `model_config`
- `training_config`

we will construct the configuration files using the provided templates. For clarity, the configuration has been divided into multiple sections, which we will review individually before merging them into a single file for the TAO Toolkit.

After examining the components, we will preview the combined training and evaluation configuration file. Note that this merged file is **not ready for use yet**, as it contains intentional placeholders and modifications that we will correct later in the workflow.

```
# DO NOT CHANGE THIS CELL
# combining configuration components in separate files and writing into one
!cat $LOCAL_SPECS_DIR/resnet18/dataset_config.txt \
    $LOCAL_SPECS_DIR/resnet18/model_config.txt \
    $LOCAL_SPECS_DIR/resnet18/training_config.txt \
    > $LOCAL_SPECS_DIR/resnet18/combined_config.txt
!cat $LOCAL_SPECS_DIR/resnet18/combined_config.txt
```

Note that we must leave an empty new line at the end of each text file to ensure the **combined_config.txt** is created properly.

Exercise 1 – Modify Dataset Config

The dataloader defines the path of the data that the model will train on and the class mapping for the classes in the dataset. We have previously generated images and masks for the training datasets. To use the newly generated training data, update the `dataset_config` parameter in the spec file to reference the correct directory.

Here are the key parameters within the `dataset_config` block:

- **dataset (str):** The input type dataset used. The currently supported dataset is **custom** to the user. Open-source datasets will be added in the future.
- **augment (bool):** If the input should be augmented online while training. When using one's own dataset to train and fine-tune a model, the dataset can be augmented while training to introduce variations. This is known as **online augmentation**. This is very useful in training as data variation improves the overall quality of the model and **prevents overfitting**.
- **augmentation_config (dict):** Configuration settings for the online augmentation process.
 - **spatial_augmentation (dict):** Supports spatial augmentation such as flip, zoom, and translate.
 - **hflip_probability (float):** Probability to flip an input image horizontally.
 - **vflip_probability (float):** Probability to flip an input image vertically.
 - **crop_and_resize_prob (float)**
 - **brightness_augmentation (dict):** Configures the color space transformation.
 - **delta (float):** Adjust brightness using a delta value.
- **input_image_type (str):** The input image type to indicate if the input image is **grayscale** or **color (RGB)**.
- **Path Strings:** These parameters define the locations of the training and validation data based on the required directory structure:
 - **train_images_path (str)**
 - **train_masks_path (str)**
 - **val_images_path (str)**
 - **val_masks_path (str)**
 - **test_images_path (str) (optional)**
- **data_class_config (dict):** A proto dictionary that contains information of training classes as part of the `target_classes` proto.
 - **target_classes (dict):** The repeated field for every training class. The following are required parameters for the `target_classes` config:
 - **name (str):** The name of the target class (e.g., "water").
 - **mapping_class (str):** The name of the mapping class for the target class. If the class needs to be trained as is, then `name` and `mapping_class` should be the same.
 - **label_id (int):** The pixel that belongs to this target class is assigned this `label_id` value in the mask image (e.g., 1 for water).

Note: The supported image extension formats for training images are “.png”, “.jpg”, “.jpeg”, “.PNG”, “.JPG”, and “.JPEG”.

```
# DO NOT CHANGE THIS CELL
# read the config file
!cat $LOCAL_SPECS_DIR/resnet18/dataset_config.txt
```

```
dataset_config { dataset: "custom" augment: true augmentation_config { spatial_augmentation { hflip_probability : 0.5
vflip_probability : 0.5 crop_and_resize_prob : 0.5 } } input_image_type: "color" train_images_path: "/workspace/tao-
experiments/data/images/train" train_masks_path: "/workspace/tao-experiments/data/masks/train"
val_images_path: "/workspace/tao-experiments/data/images/val" val_masks_path: "/workspace/tao-
experiments/data/masks/val" test_images_path: "/workspace/tao-experiments/data/images/val" data_class_config {
target_classes { name: "notflood" mapping_class: "notflood" label_id: 0 } target_classes { name: "flood" mapping_class:
"flood" label_id: 255 } } }
```

Exercise 2 – Modify Model Config

1. The segmentation model's architecture is defined using the `model_config` option in the spec file. Modifying these parameters allows users to change the backbone, input dimensions, and internal training configurations for the network.
2. Here are the key parameters within the `model_config` block:
 - **`all_projections` (bool):** For models with shortcut connections (like ResNet), this parameter defines whether all shortcuts should be instantiated with 1×1 projection layers, regardless of whether there's a change in stride between the input and output.
 - **`arch` (str):** The specific architecture of the backbone feature extractor to be used for training (e.g., `resnet`, `vgg`).
 - **`num_layers` (int):** The depth of the feature extractor for scalable architectures. For instance, for `resnet`, this defines the layer count (e.g., **18** for ResNet-18).
 - **`use_batch_norm` (bool):** A Boolean value that determines whether to use **Batch Normalization** layers in the model. Batch normalization helps stabilize and accelerate the training process.
 - **`training_precision` (dict):** Contains a nested parameter that sets the precision of the back-end training framework.
 - **`backend_floatx`:** The back-end training framework should typically be set to `FLOAT32`.
 - **`initializer` (choice):** Specifies the initialization method for convolutional layers. Supported initializations include `HE_UNIFORM`, `HE_NORMAL`, and `GLOROT_UNIFORM`.
 - **`model_input_height` (int):** The model input height dimension of the model. This value **must be a multiple of 16** for U-Net architecture to function correctly.
 - **`model_input_width` (int):** The model input width dimension of the model. This value **must also be a multiple of 16**.
 - **`model_input_channels` (int):** The model input channels dimension of the model. This should be set to **3** for a typical color image (RGB) input when using a ResNet or VGG backbone.

```
# DO NOT CHANGE THIS CELL  
# read the config file  
!cat $LOCAL_SPECS_DIR/resnet18/model_config.txt
```

```
model_config { model_input_width: 512 model_input_height: 512 model_input_channels: 3 num_layers: 18 all_projections:  
true arch: "resnet" training_precision { backend_floatx: FLOAT32 } }
```

Exercise 3 – Modify Training Config

The `training_config` describes the training and learning process using various hyperparameters. These settings directly control how the model updates its weights based on the loss function, affecting both the speed and stability of convergence.

Here are the key parameters within the `training_config` block:

- **batch_size (int):** The number of images processed per batch per GPU. A larger batch size can lead to faster training but requires more GPU memory.
- **epochs (int):** The number of times the entire dataset is passed forward and backward through the neural network. **One epoch represents one complete iteration of training through the entire dataset.**
- **log_summary_steps (int):** The interval (in steps/iterations) at which training details (like loss values) are printed to the standard output (`stdout`).
- **checkpoint_interval (int):** The number of epochs interval at which the model's checkpoint (saved weights) is saved. This allows training to be resumed if interrupted.
- **loss (str):** The loss function to be used for the segmentation task (e.g., `cross_entropy`, `dice`).
- **learning_rate (float):** The initial learning-rate value. This controls the step size used by the optimizer to update the model's weights.
- **regularizer (dict):** This parameter configures the type and weight of the regularizer used during training to prevent **overfitting**.
 - **type** (Choice): The type of the regularizer, typically `L1` or `L2`.
 - **weight** (Float): The floating-point weight assigned to the regularizer's penalty term.
- **optimizer (dict):** This parameter defines which optimization algorithm to use for training, typically **Adam**, **SGD**, or **RMSProp**.
 - **adam:** Parameters specific to the Adam optimizer, including:
 - **epsilon** (float): A very small number added to prevent any division by zero in the implementation.
 - **beta1** (float): The exponential decay rate for the first moment estimates (mean).
 - **beta2** (float): The exponential decay rate for the second moment estimates (uncentered variance).
- **activation (str):** The activation function to be used on the last layer of the model. For segmentation, the supported option is `softmax`, which provides a probability distribution over the classes for each pixel.

```
# DO NOT CHANGE THIS CELL
# read the config file
!cat $LOCAL_SPECS_DIR/resnet18/training_config.txt
```

```
training_config { batch_size: 1 epochs: 5 log_summary_steps: 5 checkpoint_interval: 5 loss: "cross_dice_sum" learning_rate:
0.0001 regularizer { type: L2 weight: 2e-5 } optimizer { adam { epsilon: 9.99999993923e-09 beta1: 0.899999976158 beta2:
0.999000012875 } } }
```

Combine Configuration Files

```
# DO NOT CHANGE THIS CELL
# combining configuration components in separate files and writing into one
!cat $LOCAL_SPECS_DIR/resnet18/dataset_config.txt \
    $LOCAL_SPECS_DIR/resnet18/model_config.txt \
    $LOCAL_SPECS_DIR/resnet18/training_config.txt \
    > $LOCAL_SPECS_DIR/resnet18/combined_config.txt
!cat $LOCAL_SPECS_DIR/resnet18/combined_config.txt
```

Initial Model Training

After preparing input data and setting up a spec file, we can start training the semantic segmentation model.

```
tao model unet train [-h] -e <EXPERIMENT_SPEC_FILE>
                    -r <RESULTS_DIR>
                    -n <MODEL_NAME>
                    -m <PRETRAINED_MODEL_FILE>
                    -k <key>
                    [-v Set Verbosity of the logger]
                    [--gpus GPUS]
                    [--gpu_index GPU_INDEX]
```

```
tao model unet train [-h] -e <EXPERIMENT_SPEC_FILE> -r <RESULTS_DIR> -n <MODEL_NAME> -m
<PRETRAINED_MODEL_FILE> -k <key> [-v Set Verbosity of the logger] [--gpus GPUS] [--gpu_index GPU_INDEX]
```

When using the `train` subtask, the `-e` argument indicates the path to the spec file, `-r` argument indicates the result directory, and the `-k` indicates the key to load the pre-trained weights. There are some arguments that might be useful such as `-n` that indicates the name of the final step model saved and `-m` to indicate the path to a pre trained model to initialize. Based on how NGC names the model downloaded, we should use `/workspace/tao experiments/unt/pretrained_resnet18/pretrained_semantic_segmentation_vresnet18` to reference the pre-trained model.

Multi-GPU support can be enabled for those with the hardware using the `--gpus` argument. When running the training with more than one GPU, we will need to modify the `batch_size` and

learning_rate. In most cases, scaling down the batch-size by a factor of `NUM_GPU's` or scaling up the learning rate by a factor of `NUM_GPUs` would be a good place to start.

```
# DO NOT CHANGE THIS CELL
# remove any previous training if exists
!rm -rf $LOCAL_EXPERIMENT_DIR/resnet18
```

While the TAO Toolkit is running, there may be some TensorFlow deprecation warning messages that can be ignored.

```
# DO NOT CHANGE THIS CELL
# train model
!tao model unet train -e $TAO_SPECS_DIR/resnet18/combined_config.txt \
-r $TAO_EXPERIMENT_DIR/resnet18 \
-n resnet18 \
-m $TAO_EXPERIMENT_DIR/pretrained_resnet18/pretrained_semantic_segmentation_vresnet18/resnet_18.hdf5 \
-k $KEY
```

Note: The training may take hours to complete. **unet** supports restarting from checkpoints in case the training job is killed prematurely. Training from the closest checkpoint may be resumed by simply re-running the **same** command.

```
# DO NOT CHANGE THIS CELL
print('Model for every epoch at checkpoint_interval mentioned in the spec file:')
print('-----')
!tree -a $LOCAL_EXPERIMENT_DIR/resnet18
```

Evaluating the Model

The model should be evaluated for its performance at the end of training. The last step model saved in the `$USER_EXPERIMENT_DIR/resnet18/weights` directory is used for evaluation, inference, and export. The evaluation metrics include **precision**, **recall**, **f1 score**, and **IOU** for every class. The evaluation also creates `results_t1t.json` as a record. We can evaluate the model with the **evaluate** subtask.

```
tao model unet evaluate [-h] -e <EXPERIMENT_SPEC_FILE> -m <MODEL_PATH> -o <OUTPUT_DIR> -k <KEY> [--
gpus GPUS] [--gpu_index GPU_INDEX]
```

		Classifier Prediction	
		Positive	Negative
Actual Value	Positive	True Positive	False Negative
	Negative	False Positive	True Negative

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

The **evaluate** subtask runs evaluation on the same validation set that was used during training. We can also run evaluation on an earlier model by editing the spec file to point to the intended model. When using the **evaluate** subtask, the **-e** argument indicates the path to the spec file, the

`-m` argument indicates the path to the model, the `-o` argument indicates where the evaluation metrics outputs should be written, and the `-k` argument indicates the key to load the model.

```
# DO NOT CHANGE THIS CELL
# evaluate the model using the same validation set as training
!tao model unet evaluate -e $TAO_SPECS_DIR/resnet18/combined_config.txt \
                        -m $TAO_EXPERIMENT_DIR/resnet18/weights/resnet18.tlt \
                        -o $TAO_EXPERIMENT_DIR/resnet18/ \
                        -k $KEY
```

To understand how the TAO Toolkit measures accuracy of the segmentation model, we'll have to understand two measures: **recall** and **precision**. The first measure is focused on identifying positive cases and is called recall. We define recall as the ability of the model to identify all true positive samples of the dataset. In mathematical terms, recall is the ratio of true positives over true positives plus false negatives. By other means, recall tells us, among all the test samples belonging to the output class, how many of them are identified correctly by the model. The next measure is called precision and is defined as the ability of the model to identify the relevant samples only. It is the ratio of true positives over true positives plus false positives. A well-known measure that summarizes the balance between precision and recall is **f1-score**, which is their harmonic mean.

Visualizing Model Inference

The `inference` subtask for `unet` may be used to visualize segmentation and generate frame-by-frame PNG format labels on a directory of images. The labels will be generated in `mask_labels_tlt`. The tool also automatically generates segmentation overlaid images in `vis_overlay_tlt`.

```
tao model unet inference [-h] -e <EXPERIMENT_SPEC_FILE> -m <MODEL_PATH> -o <OUTPUT_DIR> -k <KEY> [--
  gpus GPUS] [--gpu_index GPU_INDEX]
```

```
# DO NOT CHANGE THIS CELL
# remove any previous inference
!rm -rf $LOCAL_PROJECT_DIR/tao_infer_testing/*
```

```
# DO NOT CHANGE THIS CELL
# perform inference on the validation set
!tao model unet inference -e $TAO_SPECS_DIR/resnet18/combined_config.txt \
                        -m $TAO_EXPERIMENT_DIR/resnet18/weights/resnet18.tlt \
                        -o $TAO_PROJECT_DIR/tao_infer_testing \
                        -k $KEY
```

We can write a quick function that will help us sample random inferences. Execute the below cells to visualize the inference.

Model Export

Once we are satisfied with our model, we can move to deployment. `unet` includes an export subtask to `export` and prepare a trained U-Net model for deployment. Exporting the model decouples the training process from deployment and allows conversion to TensorRT engines outside the TAO environment. TensorRT is a highly optimized package that takes trained models and optimizes them for inference. TensorRT engines are specific to each hardware configuration and should be generated for each unique inference environment. This may be interchangeably referred to as the `.trt` or `.engine` file. The same exported TAO model may be used universally across training and deployment hardware. This is referred to as the `.etlt` file, or encrypted TAO file.

TensorRT – Programmable Inference Accelerator

NVIDIA [TensorRT](#) is a platform for high-performance deep learning inference. It includes a deep learning inference optimizer and runtime that delivers low latency and high throughput for deep learning inference applications. TensorRT-based applications perform up to **40x faster** than CPU-only platforms during inference.

With TensorRT, we can optimize neural network models trained in all major frameworks, calibrate for lower precision with high accuracy, and finally deploy to hyperscale data centers, embedded, or automotive product platforms.

How TensorRT enables optimizations on the layer graphics:

- Elimination of layers whose outputs are not used
- Fusion of convolution, bias, and ReLU operations
- Aggregation of operations with sufficiently similar parameters and the same source tensor (for example, the 1x1 convolutions in GoogleNet's inception module)
- Merging of concatenation layers by directing layer outputs to the correct eventual destination

Here are some great resources to learn more about TensorRT:

- Main Page: <https://developer.nvidia.com/tensorrt>
- Blogs: <https://devblogs.nvidia.com/speed-up-inference-tensorrt/>
- Download: <https://developer.nvidia.com/nvidia-tensorrt-download>
- Documentation: <https://docs.nvidia.com/deeplearning/sdk/tensorrt-developer-guide/index.html>
- Sample Code: <https://docs.nvidia.com/deeplearning/sdk/tensorrt-sample-support-guide/index.html>
- GitHub: <https://github.com/NVIDIA/TensorRT>
- NGC Container: <https://ngc.nvidia.com/catalog/containers/nvidia:tensorrt>

Export the Training Model

When using the `export` subtask, the `-m` argument indicates the path to the `.tlt` model file generated by `tao model unet train`, the `-e` argument indicates the path to the spec file, and `-k` argument indicates the key to *load* the model. The `-o` argument specifies the path to save the exported model to, which is defaulted to `./<input_file>.onnx`. There is an optional argument, `--gen_ds_config`, that are useful for us. The `--gen_ds_config` argument indicates whether to generate a template inference configuration file as well as a label file.

```
tao model unet export [-h] -e <EXPERIMENT_SPEC> -m <MODEL> -k <KEY> [-o OUTPUT_PATH] [--engine_file  
ENGINE_FILE] [--gen_ds_config] [--gpus GPUS] [--gpu_index GPU_INDEX]
```

```
# DO NOT CHANGE THIS CELL  
# remove any previous exports if exists  
!rm $LOCAL_EXPERIMENT_DIR/resnet18/weights/resnet18.onnx
```

```
# DO NOT CHANGE THIS CELL  
# export model  
!tao model unet export -m $TAO_EXPERIMENT_DIR/resnet18/weights/resnet18.tlt \  
-e $TAO_SPECS_DIR/resnet18/combined_config.txt \  
-k $KEY \  
--gen_ds_config
```

Generate TensorRT Engine

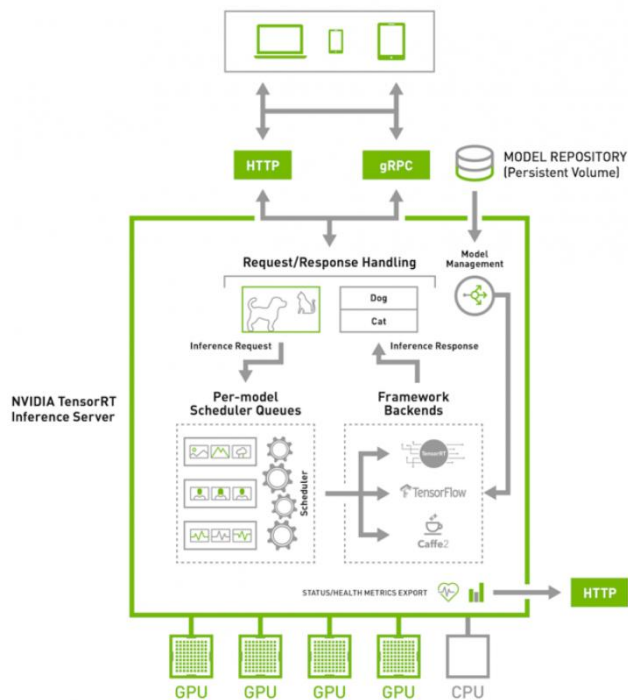
The `.onnx` file generated from `tao model unet export` is taken as an input to `tao deploy unet` to generate an optimized TensorRT engine. When using the `gen_trt_engine` subtask, the `-m` argument indicates the path to the `.onnx` model file to be exported, the `-e` argument indicates the path to the spec file, the `-k` argument indicates the key to load the model, the `--engine_file` argument specifies the path to the serialized TensorRT engine file, and the `-r` argument indicates the directory where the JSON status-log file will be dumped. There is an optional argument, `--max_batch_size`, that is useful for us. The `---max_batch_size` argument indicates the maximum batch size of TensorRT engine, which is defaulted to `1`.

Note that the TensorRT file is hardware specific and cannot be generalized across GPUs. Since a TensorRT engine file is hardware specific, you cannot use an engine file for deployment unless the deployment GPU is identical to the training GPU. This is true in our case since the Triton Inference Server will run on the same hardware.

```
tao deploy unet gen_trt_engine [-h] [-v] -m <MODEL_PATH> -e <EXPERIMENT_SPEC> -r <RESULTS_DIR> [-k KEY] [--  
max_batch_size MAX_BATCH_SIZE] [--gpu_index GPU_INDEX]
```

```
# DO NOT CHANGE THIS CELL
# remove any previous engine if exists
!rm $LOCAL_EXPERIMENT_DIR/export/resnet18.engine
```

```
# DO NOT CHANGE THIS CELL
# generate TensorRT engine
!tao deploy unet gen_trt_engine -m $TAO_EXPERIMENT_DIR/resnet18/weights/resnet18
                                -e $TAO_SPECS_DIR/resnet18/combined_config.txt \
                                -r $TAO_EXPERIMENT_DIR/export/ \
                                -k $KEY \
                                --engine_file $TAO_EXPERIMENT_DIR/export/resnet1
                                --max_batch_size 8
```



Note from the logs that the input layer is `input_1:0` and the output layer is `argmax_1`.

Chapter 3: Deploying a Model for Inference

Introduction

In this chapter, we transition from model development to **model deployment**. After training the flood-segmentation network, the next step is to make it usable in real-world systems. To accomplish this, we deploy the model using the **NVIDIA Triton Inference Server**, a high-performance inference platform designed for production environments.

This section explains:

- how to prepare the **model repository** structure required by Triton,
- how to create the necessary **configuration files**, and
- how to send inference requests to the server once the model is deployed.

By the end of this chapter, you will understand how a trained model moves from experimentation to an operational inference pipeline—an essential step in building a practical flood-risk monitoring system.

1. Model Deployment

Model deployment refers to the integration of deep learning inference into an existing system or process. Sometimes it's advantageous to keep the deep learning model isolated, and other times, it needs to be more available for broad use. We imagine our flood detection model to have more utility if it can produce real-time inference, therefore we employ a solution to allow for its use via network APIs. While model training focuses on accuracy as the key metric, model deployment concentrates on inference performance metrics such as [throughput](#), [latency](#), and hardware utilization.

Model deployment considerations:

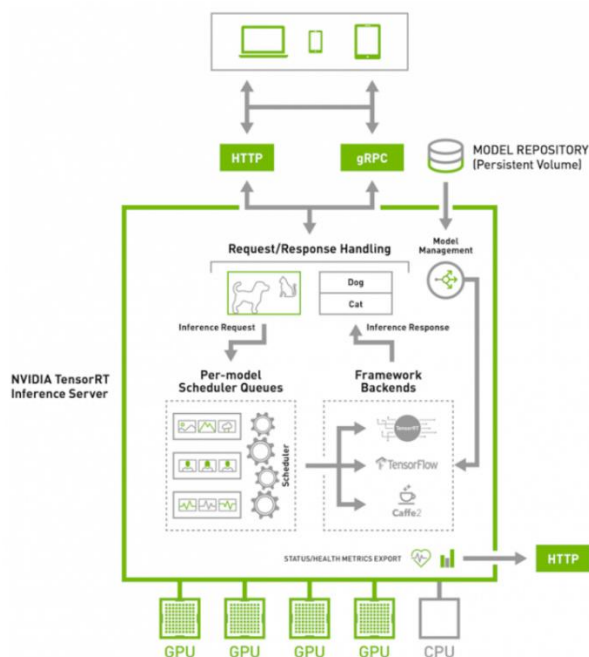
- **Throughput**: the amount of data that are processed in a specific amount of time
- **Latency**: the amount of time it takes for the input to be processed is critical for real-time inference
- **Hardware Utilization**: how much of the hardware is being utilized/unutilized
- **Deployment Environment**: environments can vary depending on the application - public cloud, on-premises core (data center), enterprise edge, and on embedded devices.

In the context of a disaster risk monitoring system, we should consider optimizing throughput at a low cost for processing satellite images to create high fidelity maps.

2. Introduction to Triton Inference Server

NVIDIA [Triton Inference Server](#) simplifies the deployment of AI models at scale in production. It's an open-source, inference-serving software that lets teams deploy trained AI models from any

framework, from local storage, or from cloud service providers on any GPU or CPU-based infrastructure, cloud, data center, or edge. The below figure shows the Triton Inference Server's high-level architecture. The model repository is a *file-system based repository* of the models that Triton Inference Server will make available for inferencing. Inference requests arrive at the server via either [HTTP/REST](#), [gRPC](#), or by the C API and are then routed to the appropriate per-model scheduler. Triton Inference Server implements multiple scheduling and batching algorithms that can be configured on a model-by-model basis. Each model's scheduler optionally performs batching of inference requests and then passes the requests to the backend corresponding to the model type. The backend performs inferencing using the inputs provided in the batched requests to produce the requested outputs. The outputs are then returned.



Server

Deploying a model with Triton requires two main components: the **server software** and the **client tools** used to send requests. The easiest way to begin is by using the official Triton Inference Server container available through the **NVIDIA NGC catalog**. This container includes all necessary dependencies and provides a consistent environment for running high-performance inference workloads.

Developers who need deeper customization or prefer to build Triton from source can refer to the official [QuickStart](#) and [Build documentation](#), which outline both container-based and manual installation workflows.

Client

To interact with the deployed model, Triton provides **client libraries** for both Python and C++. These libraries offer convenient APIs for sending inference requests and managing

communication with the Triton Inference Server. Applications can use either **HTTP/REST** or **gRPC** protocols to access all major server functions, including:

- performing model inference
- checking server status and health
- retrieving performance metrics
- managing models within the repository

In addition, the client libraries support both **system memory** and **CUDA shared memory**, allowing efficient transfer of inputs and outputs—especially useful for GPU-accelerated workloads.

Developers can download or build the Triton client libraries using the official documentation, which also includes examples demonstrating their usage in Python and C++.

3. Model Repository

Triton Inference Server serves models within a model repository. When you first run Triton Inference Server, you'll specify the model repository where the models reside:

```
tritonserver --model-repository=/models
```

Inside the model repository, **each model is stored in its own folder**, and every folder represents a distinct model. For example, to deploy the flood segmentation network, we place it in a directory such as:

```
root@server:/models$ tree
.
├── flood_segmentation_model
│   ├── 1
│   │   └── model.plan
│   └── config.pbtxt
└── flood_segmentation_model_batch
    ├── 1
    │   └── model.plan
    └── config.pbtxt
```

Every model directory follows a consistent structure. At the top level, a **configuration file** named `config.pbtxt` defines key details such as:

- model backend (TensorRT, PyTorch, ONNX, TensorFlow, etc.)
- expected input dimensions
- maximum batch size
- output specifications
- optimization or execution settings

Triton also supports **model versioning**. Each version is stored in a numbered subfolder, beginning with `1`.

This versioned folder contains the actual model files, for example, an ONNX file or a TensorRT engine.

This structure enables clean version control, smooth model updates, and predictable deployment behavior. More information on model repositories and directory conventions can be found in the official Triton Inference [Server documentation](#).

We can also add a file representing the names of the outputs. We have omitted this step in this chapter for the sake of brevity. For more details on how to work with model repositories and model directory structures in Triton Inference Server, please see the [documentation](#). Below we'll create the model directory structure for our flood detection segmentation model and copy the exported TensorRT engine into the model repository. For demonstration purposes, we have prepared a sample ResNet18 model (`sample_resnet18.engine`).

```
# DO NOT CHANGE THIS CELL
# set environment variables
import os

%set_env LOCAL_PROJECT_DIR=/dli/task/tao_project
%set_env LOCAL_DATA_DIR=/dli/task/flood_data

os.environ["LOCAL_EXPERIMENT_DIR"]=os.path.join(os.getenv("LOCAL_PROJECT_DIR"), "unet")
```

```
# DO NOT CHANGE THIS CELL
# create directory for model
!mkdir -p models/flood_segmentation_model/1

# copy sample_resnet18.engine to the model repository
!cp $LOCAL_EXPERIMENT_DIR/export/sample_resnet18.engine models/flood_segmentation_model/1/model.plan
```

Exercise 1 – Model Configuration

With the model directory prepared, the next step is to define the configuration that Triton will use when loading and executing the model. Your task in this exercise is to create the `config.pbtxt` file based on the following requirements:

A minimal configuration must include:

- the **model name**
- the **platform or backend** (such as TensorRT, ONNX, or PyTorch)
- the **maximum batch size**
- the **definitions of all input tensors** (names, data types, and shapes)
- the **definitions of all output tensors**

For this project, the required output tensor name can be found in the `nvinfer_config.txt` file under the `output-blob-names` entry.

This exercise helps reinforce how Triton interprets and manages models internally. For additional configuration options, such as optimization settings or batching behavior, refer to the official Triton Inference Server [documentation](#).

```
configuration = """
name: "flood_segmentation_model"
platform: "tensorrt_plan"
max_batch_size: 1
input: [
  {
    name: "input_1"
    data_type: TYPE_FP32
    format: FORMAT_NCHW
    dims: [ 3, 512, 512 ]
  }
]
output: {
  name: "argmax_1"
  data_type: TYPE_INT32
  dims: [ 512, 512, 1 ]
}
"""

with open('models/flood_segmentation_model/config.pbtxt', 'w') as file:
    file.write(configuration)
```

```
# DO NOT CHANGE THIS CELL
# show model repository folder structure
!tree -a models
```

```
models
├── flood_segmentation_model
│   ├── 1
│   │   └── model.plan
│   └── config.pbtxt
```

2 directories, 2 files

4. Run Inference on Triton Inference Server

With our model directory structures created, models defined and exported, and configuration files created, we will now wait for Triton Inference Server to load our models. We have set to use Triton Inference Server in **polling** mode. This means that Triton Inference Server will continuously poll for modifications to our models or for newly created models - once every 30 seconds. Please run the cell below to allow time for Triton Inference Server to poll for new models/modifications before proceeding. Due to the asynchronous nature of this step, we have added 15 seconds to be safe.

Server Health Status

```
# DO NOT CHANGE THIS CELL
!sleep 45
```

At this point, our models should be deployed and ready to use! To confirm Triton Inference Server is up and running, we can send a `curl` request to the below URL. The HTTP request returns status `200` if Triton is ready and `non-200` if it is not ready. We can also send a `curl` request to our model endpoints to confirm our models are deployed and ready to use. Additionally, we will also see information about our models such:

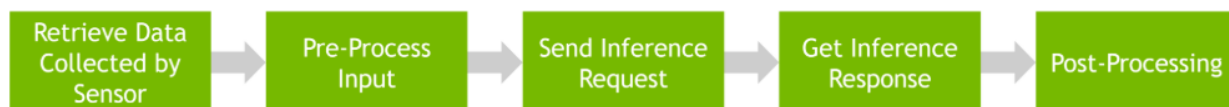
- The name of our model
- The versions available for our model
- The backend platform (e.g. `tensorrt`, `pytorch_libtorch`, `onnxruntime_onnx`)
- The inputs and outputs, with their respective names, data types, and shapes

```
# DO NOT CHANGE THIS CELL
!curl -v triton:8000/v2/health/ready
```

```
# DO NOT CHANGE THIS CELL
!curl -v triton:8000/v2/models/flood_segmentation_model
```

5. Send Inference Request to Server

With our models deployed, it is now time to send inference requests to our models. Triton Inference Server itself does not do anything with your input tensors, it simply feeds them to the model. Same for outputs. Ensuring that the pre-processing operations used for inference are defined identically as they were when the model was trained is key to achieving high accuracy.



Exercise 2 – Pre-Process Inputs

When we developed our segmentation model with the TAO Toolkit, some data pre-processing was done for training. Therefore, we need to perform the same normalization and mean subtraction to produce the final float planar data that the TensorRT engine is looking for. We can get the `offsets` and `net-scale-factor` from the `nvinfer_config.txt` [file](#). The pre-processing function is:

$$y = \text{net scale factor} * (x - \text{mean})$$

where:

- **x** is the input pixel value. It is an int8 with range [0,255].
- **mean** is the corresponding mean value, read either from the mean file or as `offsets[c]`. Here `c` is the channel to which the input pixel belongs and `offsets` is the array specified in the configuration file. In the case of `sample_resnet18.engine`, it is `127.5`.
- **net-scale-factor** is the pixel scaling factor specified in the configuration file. In the case of `sample_resnet18.engine`, it is `1/127.5`, or `0.00784313725490196`.
- **y** is the corresponding output pixel value. It is a float.

```
# DO NOT CHANGE THIS CELL
# import dependencies
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np
import time
import warnings
import random

warnings.filterwarnings("ignore")
```

```
# define pre-process function
def preprocess_image(image):
    """
    This function returns the pre-processed image as a NumPy array.
    """
    # Load image as a 32-bit floating point array
    image_ary=np.asarray(image)
    image_ary=image_ary.astype(np.float32)

    # pre-processing
    image_ary=(image_ary-127.5)*0.00784313725490196

    # unet segmentation model requires the data to be in BGR format
    BGR=np.empty_like(image_ary)
    BGR[:, :, 0]=image_ary[:, :, 2]
    BGR[:, :, 1]=image_ary[:, :, 1]
    BGR[:, :, 2]=image_ary[:, :, 0]
    image_ary=BGR

    # convert array from h, w, c to 1, c, h, w
    image_ary=np.transpose(image_ary, [2, 0, 1])
    image_ary=np.expand_dims(image_ary, axis=0)
    return image_ary
```

```
# DO NOT CHANGE THIS CELL
# choose random image
random_image_file=random.sample(os.listdir(os.path.join(os.getenv('LOCAL_DATA_DIR'), 'images', 'all_images')), 1)[0]

# preprocess
image=Image.open(os.path.join(os.getenv('LOCAL_DATA_DIR'), 'images', 'all_images', random_image_file))
mask=Image.open(os.path.join(os.getenv('LOCAL_DATA_DIR'), 'masks', 'all_masks', random_image_file))
image_ary=preprocess_image(image)

print('The input array has a shape of {}'.format(image_ary.shape))
```

The input array has a shape of (1, 3, 512, 512).

Next, we'll load the `tritonclient.http` module. We will also define the input and output names of our model, the name of our model, the URL where our models are deployed with Triton Inference Server (in this case the host `triton:8000`), and our model version.

```
# DO NOT CHANGE THIS CELL
import tritonclient.http as tritonhttpclient
from pprint import pprint

# set parameters
VERBOSE=False
input_name='input_1'
input_shape=(1, 3, 512, 512)
input_dtype='FP32'
output_name='softmax_1'
model_name='flood_segmentation_model'
url='triton:8000'
model_version='1'
```

We'll instantiate our client `triton_client` using the `tritonhttpclient.InferenceServerClient` class and access the model metadata with the `get_model_metadata` method as well as get our model configuration with the `get_model_config` method.

```
# DO NOT CHANGE THIS CELL
# instantiate Triton Inference Server client
triton_client=tritonhttpclient.InferenceServerClient(url=url, verbose=VERBOSE)

# get model metadata
print('-----Metadata-----')
model_metadata=triton_client.get_model_metadata(model_name=model_name, model_version=model_version)
pprint(model_metadata)

# get model configuration
print('-----Configuration-----')
model_config=triton_client.get_model_config(model_name=model_name, model_version=model_version)
pprint(model_config)
```

```
-----Metadata-----
{'inputs': [{'datatype': 'FP32',
              'name': 'input_1',
              'shape': [-1, 3, 512, 512]}],
 'name': 'flood_segmentation_model',
 'outputs': [{'datatype': 'FP32',
              'name': 'softmax_1',
              'shape': [-1, 512, 512, 2]}],
 'platform': 'tensorrt_plan',
 'versions': ['1']}
-----Configuration-----
```

```
{'backend': '',
  'batch_input': [],
  'batch_output': [],
  'cc_model_filenames': {},
  'default_model_filename': 'model.plan',
  'input': [{'allow_ragged_batch': False,
              'data_type': 'TYPE_FP32',
              'dims': [3, 512, 512],
              'format': 'FORMAT_NCHW',
              'is_shape_tensor': False,
              'name': 'input_1'}],
  'instance_group': [{'count': 1,
                      'gpus': [0],
                      'host_policy': '',
                      'kind': 'KIND_GPU',
                      'name': 'flood_segmentation_model',
                      'passive': False,
                      'profile': [],
                      'secondary_devices': []}],
  'max_batch_size': 1,
  'metric_tags': {},
  'model_warmup': [],
  'name': 'flood_segmentation_model',
  'optimization': {'eager_batching': False,
                   'gather_kernel_buffer_threshold': 0,
                   'input_pinned_memory': {'enable': True},
                   'output_pinned_memory': {'enable': True},
                   'priority': 'PRIORITY_DEFAULT'},
  'output': [{'data_type': 'TYPE_FP32',
              'dims': [512, 512, 2],
              'is_shape_tensor': False,
              'label_filename': '',
              'name': 'softmax_1'}],
  'parameters': {},
  'platform': 'tensorrt_plan',
  'version_policy': {'latest': {'num_versions': 1}}}
```

We'll instantiate a placeholder for our input data using the expected input name, shape, and data type. We'll set the data of the input to be the NumPy array representation of our image. We'll also instantiate a placeholder for our output data using just the output name. Lastly, we'll submit our input to the Triton Inference Server using the `triton_client.infer` method, specifying our model name, model version, inputs, and outputs and convert our result to a NumPy array.

```
# DO NOT CHANGE THIS CELL
inference_input=tritonhttpclient.InferInput(input_name, input_shape, input_dtype)
output=tritonhttpclient.InferRequestedOutput(output_name)

inference_input.set_data_from_numpy(image_ary)

# time the process
start=time.time()
response=triton_client.infer(model_name,
                             model_version=model_version,
                             inputs=[inference_input],
                             outputs=[output])

latency=time.time()-start
logits=response.as_numpy(output_name)

print('The output array has a shape of {}'.format(logits.shape))
print('It took {} per inference.'.format(round(latency, 3)))
```


The output array has a shape of (1, 512, 512, 2).
It took 2.058 per inference.

Visualize Results

Since the last layer of our segmentation model is a [SoftMax](#) layer, we can identify the largest logit value per pixel and visually confirm that our model inferred the presence of flood.

```
# DO NOT CHANGE THIS CELL
# visualize results
fig, ax_arr=plt.subplots(1, 3, figsize=[15, 5], sharex=True, sharey=True)
ax_arr[0].set_title('Input Data')
ax_arr[1].set_title('Inference')
ax_arr[2].set_title('Actual')
ax_arr[0].set_xticks([])
ax_arr[0].set_yticks([])

ax_arr[0].imshow(image)
ax_arr[1].imshow(np.argmax(logits, axis=3)[0]*255, cmap='gray')
ax_arr[2].imshow(mask, cmap='gray')

fig.tight_layout()
plt.show()
```



Recall that we used a random input image for demonstration. In case you don't get an image that shows presence of flood, please feel free to re-run the inference starting from Exercise 2.

We can iterate through all input images to see how quickly Triton is able to perform inference.

```
# DO NOT CHANGE THIS CELL
time_list=[]

for image_path in os.listdir(os.path.join(os.getenv('LOCAL_DATA_DIR'), 'images', 'all_images')):
    image=Image.open(os.path.join(os.getenv('LOCAL_DATA_DIR'), 'images', 'all_images', image_path))
    image_ary=preprocess_image(image)
    inference_input.set_data_from_numpy(image_ary)

    # time the process
    start=time.time()
    response=triton_client.infer(model_name,
                                model_version=model_version,
                                inputs=[inference_input],
                                outputs=[output])
    time_list.append(time.time()-start)
    logits=response.as_numpy(output_name)

latency=sum(time_list)/len(time_list)
print('It took {} seconds to infer {} images.'.format(round(sum(time_list), 3), len(time_list)))
print('On average it took {} seconds per inference.'.format(round(latency, 3)))
```

Run Batch Inference

One factor to consider for inference optimization is batch size, or how many samples you process at once. Below we're going to perform inference in batches.

```
# DO NOT CHANGE THIS CELL
batch_size=8
```

```
# DO NOT CHANGE THIS CELL
# create directory for model
!mkdir -p models/flood_segmentation_model_batch/1

# copy sample_resnet18.engine to the model repository
!cp $LOCAL_EXPERIMENT_DIR/export/sample_resnet18.engine models/flood_segmentation_model_batch/1/model.plan
```

```
# DO NOT CHANGE THIS CELL
configuration = """
name: "flood_segmentation_model_batch"
platform: "tensorrt_plan"
max_batch_size: {}
input: [
  {{
    name: "input_1"
    data_type: TYPE_FP32
    format: FORMAT_NCHW
    dims: [ 3, 512, 512 ]
  }}
]
output: {{
  name: "softmax_1"
  data_type: TYPE_FP32
  dims: [ 512, 512, 2]
}}
""".format(batch_size)

with open('models/flood_segmentation_model_batch/config.pbtxt', 'w') as file:
    file.write(configuration)
```

```
# DO NOT CHANGE THIS CELL
# show model repository folder structure
!tree -a models
```

```
# DO NOT CHANGE THIS CELL
!sleep 45
```

```
# DO NOT CHANGE THIS CELL
!curl -v triton:8000/v2/models/flood_segmentation_model_batch
```

```

# DO NOT CHANGE THIS CELL
# define new input shape
batch_input_shape=(batch_size, 3, 512, 512)

batch_inference_input=tritonhttpclient.InferInput(name='input_1', shape=batch_input_shape, datatype='FP32')
batch_output=tritonhttpclient.InferRequestedOutput('softmax_1')

# create empty array for the batch input
batch_ary=np.empty(batch_input_shape).astype(np.float32)

time_list=[]

# iterate through all images
for idx, image_path in enumerate(os.listdir(os.path.join(os.getenv('LOCAL_DATA_DIR'), 'images', 'all_images'))):
    image=Image.open(os.path.join(os.getenv('LOCAL_DATA_DIR'), 'images', 'all_images', image_path))
    batch_ary[idx%batch_size]=preprocess_image(image)
    if idx%batch_size==(batch_size-1):
        batch_inference_input.set_data_from_numpy(batch_ary)

        # time the process
        start=time.time()
        response=triton_client.infer(model_name='flood_segmentation_model_batch',
                                     model_version='1',
                                     inputs=[batch_inference_input],
                                     outputs=[batch_output])
        time_list.append(time.time()-start)
        logits=response.as_numpy(output_name)

batch_latency=sum(time_list)/len(time_list)
print('It took {} seconds to infer {} images.'.format(round(sum(time_list), 3), len(time_list)*batch_size))
print('On average it took {} seconds per inference.'.format(round(batch_latency, 3)))

```

Inference Performance Optimization

There is usually a trade-off between latency and throughput. Finding the right balance between them usually involves satisfying the application requirements. As an example, the Triton Inference Server can be connected to front-end applications such as those that power <https://www.balcony.io/>, which provides an emergency management platform. Amongst other services, it has the ability to send alerts to personal devices in real-time. NVIDIA's GPUs deliver high throughput at higher batch size. However, for real-time applications, the real constraint on services isn't batch size or even throughput, but rather the latency required to deliver an outstanding experience for end users. You can read more about how Triton Inference Server is used to enable rapid disaster detection and response [here](#).

```

# DO NOT CHANGE THIS CELL
# plot throughput vs. latency
plt.title('Inference Performance Comparison')
plt.plot([latency, batch_latency], [1/latency, batch_size/batch_latency], marker='o')
plt.text(latency, (1/latency)-2.5, 'Non-Batching')
plt.text(batch_latency, (batch_size/batch_latency)-2.5, 'Batching ({}).format(batch_size))

plt.xlabel('Latency (Second)')
plt.ylabel('Throughput (Image/Second)')
plt.xlim(xmin=0, xmax=max(batch_latency, latency)*1.25)
plt.ylim(ymin=0, ymax=max(1/latency, batch_size/batch_latency)*1.25)
plt.show()

```

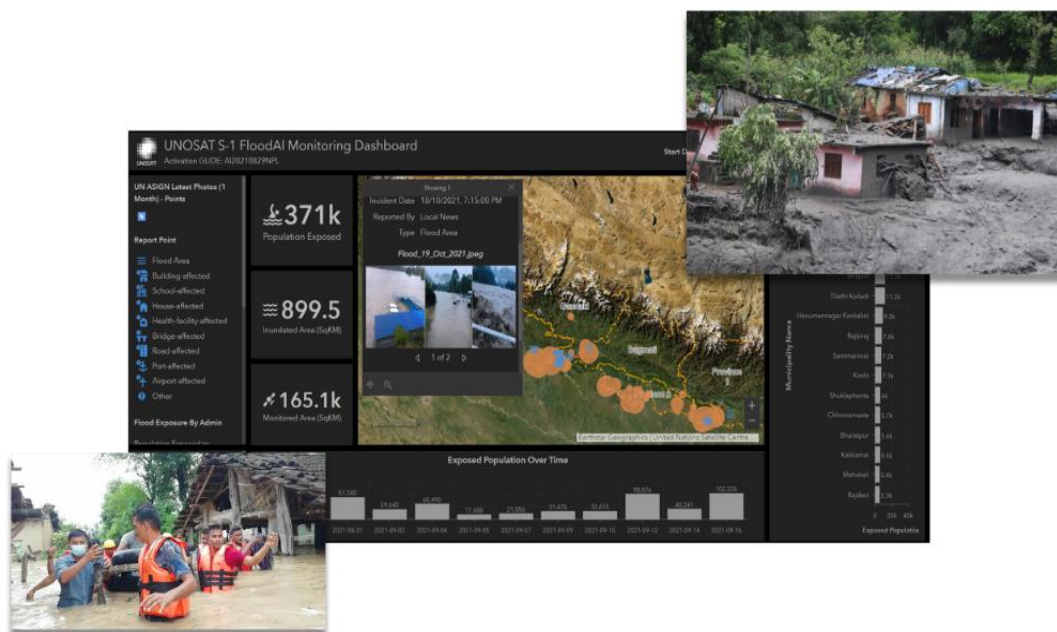
Chapter 4: UNOSAT Flood Impact Analysis Case Study

Introduction

In this chapter we will learn how UNOSAT, the United Nations Satellite Centre, uses satellite imagery and deep-learning based flood detection models to support emergency response efforts.

Background

Nepal is a hazard-prone country. Recurring disasters have been monitored throughout the years, including severe floods in 2008, 2014, and 2017. In 2015, following the 7.8-magnitude earthquake, UNOSAT's Emergency Mapping service was activated to provide satellite-derived damage assessment in support of the humanitarian response efforts on the ground. Acting as coordinator for mapping products, UNOSAT integrated the results from various sources in a publicly available web map. Very innovative at the time, this online viewing tool gathered all the analysis results. It also showed images collected from the ground through the newly developed UN ASIGN app. With the crowdsourcing app, individuals are able to send geotagged content, including photos of damages, to validate the satellite imagery-based assessment. Over the years, UNOSAT has worked with different partners in Nepal, providing technical support and implementing different activities to strengthen capacities in using Geospatial Information Technology (GIT) and data for Disaster Risk Management (DRM).



Nepal June 2021 Flood Event Case Study

In a more recent incident that occurred during June 2021, Nepal was hit by heavy rainfall. Flooding and landslides were reported in many parts of the country and affected thousands of families^[1]. Later in August of the same year, rainfall intensified again for several consecutive days, resulting in rivers overflowing, landslides triggered across the hills, and widespread inundation in the southern plains. In the affected areas, many houses, informal settlements, and other buildings were destroyed, forcing people to seek refuge with relatives or in temporary shelters. Health and sanitation concerns arose as access to running water became limited, increasing the risk of water-borne disease transmission, and limiting the ability to adhere to preventive measures against COVID-19.

[1] <https://reliefweb.int/disaster/fl-2021-000134-npl>

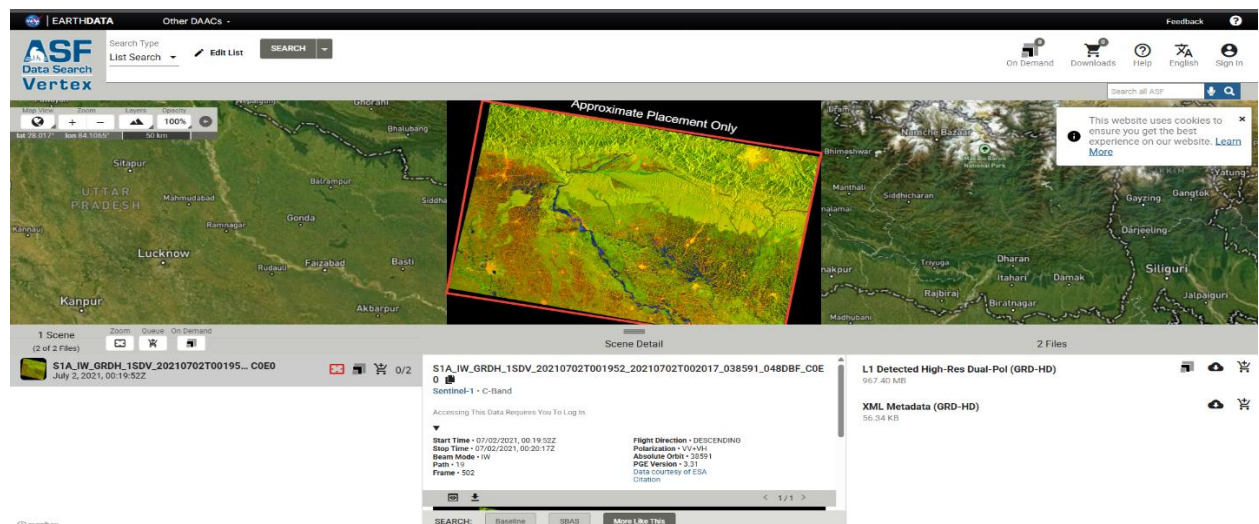
To rapidly respond, UNOSAT Rapid Mapping Service deployed the AI flood detection dashboard (shown below). Flood impact statistics such as sqkm of flood and flood affected population by admin units were automatically generated and displayed on the dashboard. UNOSAT FloodAI was able to process all the available Sentinel-1 overlapping with the area of interested shared by the Information Management Unit of the United Nation Resident Coordination Office in Nepal during the 29 days of activation. We will demonstrate the use of the flood segmentation model on Sentinel-1 images captured around July 2nd, 2021, to analyze the impact of the flood.

Sentinel-1 Data

Sentinel-1 images can be viewed freely from the Alaska Satellite Facility (ASF) Data Search Vertex. Below are the instructions:

1. Open the [ASF Data Search Vertex](#)
2. Input the image

name: **S1A_IW_GRDH_1SDV_20210702T001952_20210702T002017_038591_048DBF_C0E0** under "List of Scene Name". You can also use [this link](#).



3. Find the image name in the bottom left corner
4. Click "**Zoom to scene**"

Note:- No ASF login required.

Load Data

The Sentinel-1 images have been downloaded and pre-processed using the [ESA SNAP toolbox](#). Below, we will load the two polarizations of Sentinel-1, namely VV and VH, and combine them into an RGB image.

```
# DO NOT CHANGE THIS CELL
# import dependencies
import matplotlib.pyplot as plt
import os
import numpy as np
import matplotlib.image as mpimg
import warnings
import gc
import random

warnings.filterwarnings('ignore')
```

First, we need to unzip the **UNOSAT_case_study.zip** file, which includes all of the data.

```
# DO NOT CHANGE THIS CELL
# unzip data file
!unzip data/UNOSAT_case_study.zip -d UNOSAT_case_study
```

```
Archive: data/UNOSAT_case_study.zip
  creating: UNOSAT_case_study/baseline_maps/
  inflating: UNOSAT_case_study/baseline_maps/permanent_water_sample.PNG
```

Visualize Data

The Sentinel-1 images are divided into tiles of size 512 x 512 so that they are easier to manipulate (as opposed to one large file).

```
# DO NOT CHANGE THIS CELL
# define function to create RGB composite from VV and VH bands
def RGB_composite(tile_name):
    vv=mpimg.imread(os.path.join(os.getenv('NEPAL_VV_PATH'), tile_name))
    red=np.clip(vv, 0, 1)

    vh=mpimg.imread(os.path.join(os.getenv('NEPAL_VH_PATH'), tile_name))
    green=np.clip(vh, 0, 1)

    blue=np.clip((vv-vh), 0, 1)
    stacked=np.dstack((red, green, blue))
    return stacked
```



```

# DO NOT CHANGE THIS CELL
# display the RGB composite of one tile
tile_name=random.choice(os.listdir(os.getenv('NEPAL_VV_PATH')))
print(f'Displaying {tile_name}.')

# create RGB composite
RGB_tile=RGB_composite(tile_name)
cmaps=['Reds', 'Greens', 'Blues']

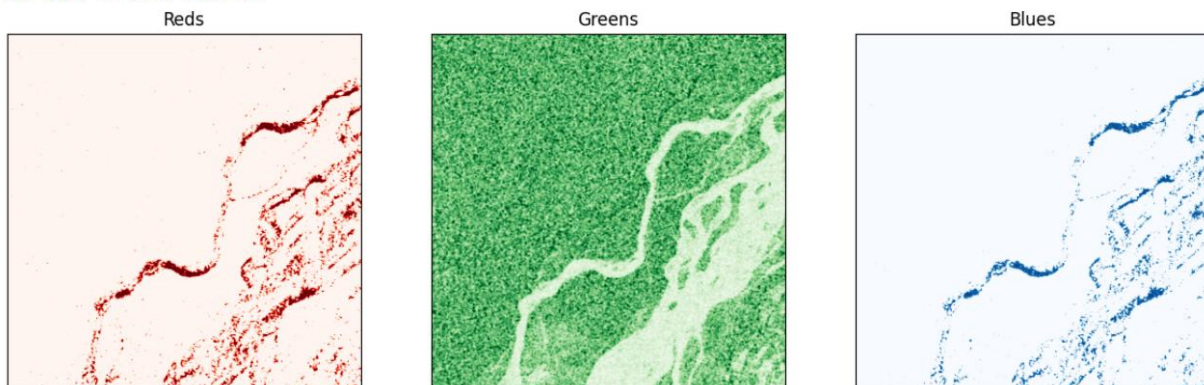
# plot RGB bands
fig, ax=plt.subplots(1, 3, figsize=(15, 5))

for idx in range(3):
    ax[idx].imshow(RGB_tile[:, :, idx], cmap=cmaps[idx])
    ax[idx].get_yaxis().set_visible(False)
    ax[idx].get_xaxis().set_visible(False)
    ax[idx].set_title(cmaps[idx])

print(f'The shape is {RGB_tile.shape}.')
plt.show()

```

Displaying tile_22_51.png.
The shape is (512, 512, 3).



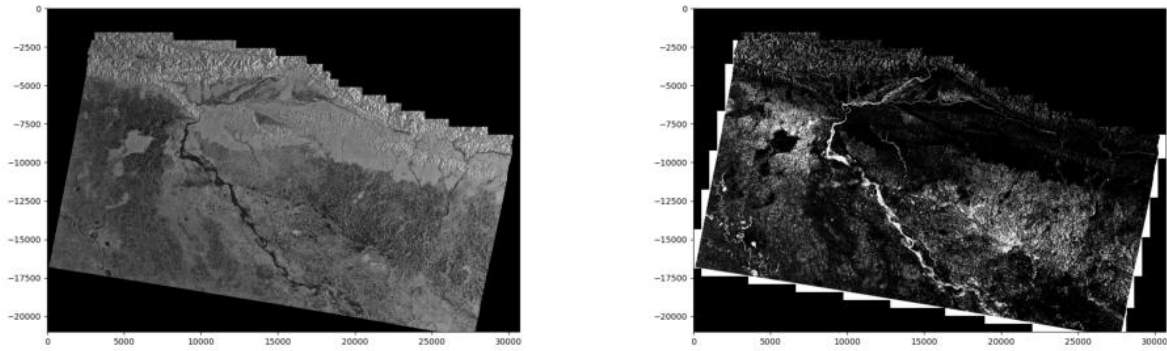
```

# DO NOT CHANGE THIS CELL
# plot composite for all tiles
fig=plt.figure(figsize=(10, 10))
ax=plt.subplot(111)
max_x, max_y=0, 0

for each_tile in os.listdir(os.getenv('NEPAL_VH_PATH')):
    if (each_tile.split('.')[-1]=='png'):
        tile_name=each_tile.replace('.png', '')
        _, y, x=tile_name.split('_')
        y=int(y)
        x=int(x)
        if y>max_y:
            max_y=y
        if x>max_x:
            max_x=x
        image=mpimg.imread(os.path.join(os.getenv('NEPAL_VH_PATH'), each_tile))
        plt.imshow(image, extent=(512*x, 512*(x+1), -512*(y+1), -512*y), cmap='gray')

ax.set_xlim([0, max_x*512])
ax.set_ylim([-max_y*512, 0])
ax.set_facecolor("black")
plt.show()

```

Processing large image files is memory intensive. To ensure we have sufficient memory for the rest of the notebook, we free up some memory with `del()` and `gc.collect()`.

DO NOT CHANGE THIS CELL

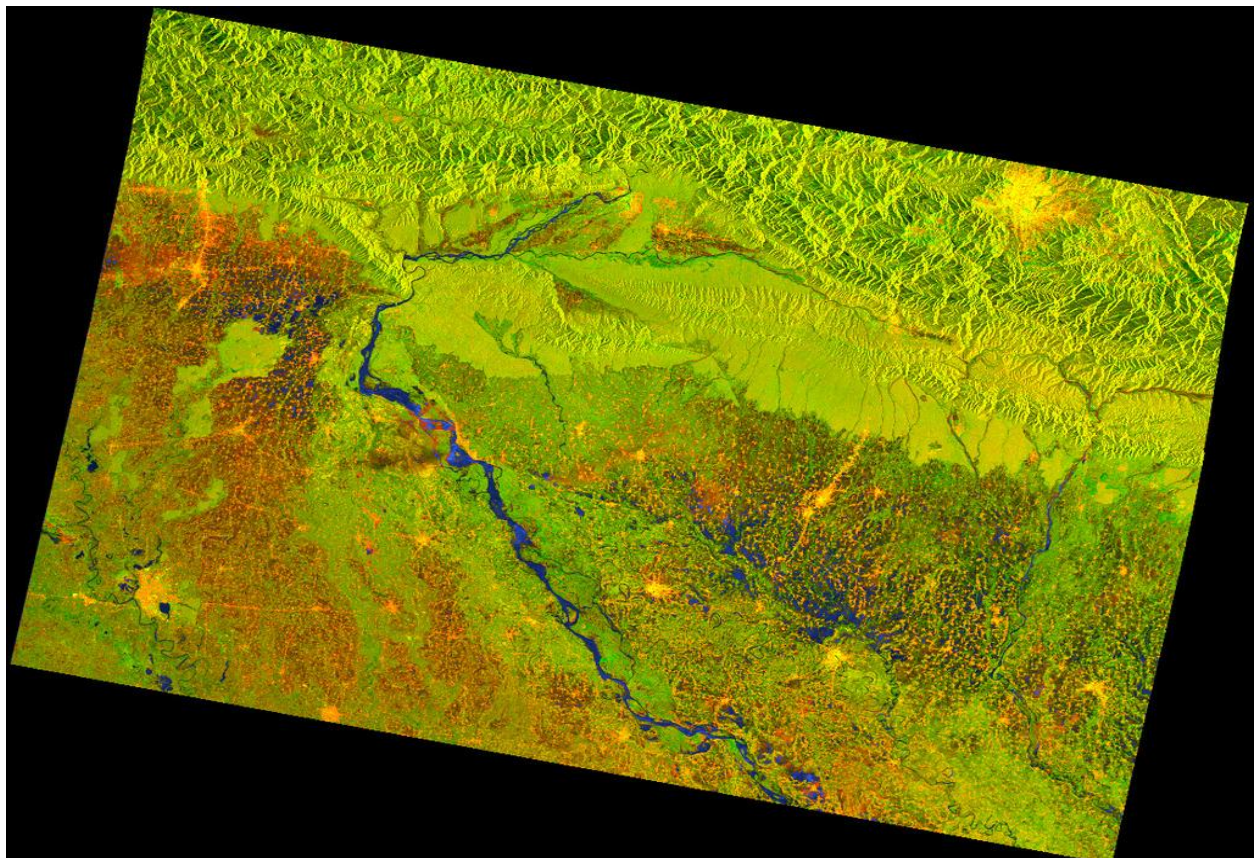
`del(fig)`

`del(ax)`

`gc.collect()`

`!free`

	total	used	free	shared	buff/cache	available
Mem:	28752064	6352548	271040	342112	22128476	21635876
Swap:	0	0	0			



You can compare the composite with one that is generated on ASF.

Segmentation Model Inference

For inference, we will send requests to the Triton Inference Server to ensure the best performance. We will use the previously trained `sample_resnet18.engine` TensorRT engine, which has been optimized for inference performance.

```
# DO NOT CHANGE THIS CELL
!curl -v triton:8000/v2/health/ready
```

```
# DO NOT CHANGE THIS CELL
import tritonclient.http as tritonhttpclient

# set parameters
VERBOSE=False
input_name='input_1'
input_shape=(1, 3, 512, 512)
input_dtype='FP32'
output_name='softmax_1'
model_name='flood_segmentation_model'
url='triton:8000'
model_version='1'
```

```
# DO NOT CHANGE THIS CELL
# configure Triton client
triton_client=tritonhttpclient.InferenceServerClient(url=url, verbose=VERBOSE)
model_metadata=triton_client.get_model_metadata(model_name=model_name, model_version=model_version)
model_config=triton_client.get_model_config(model_name=model_name, model_version=model_version)
```

We will performance inference on the image tiles. Since we are only interested in a specific area, we limit this to a subset of the total tiles.

```
# DO NOT CHANGE THIS CELL
# define function to pre-process images for Triton consumption
def get_image_ary(tile_name):
    image=RGB_composite(tile_name)
    image_ary=((image*255)-127.5)*0.00784313725490196
    image_ary=np.transpose(image_ary, [2, 0, 1])
    image_ary=np.expand_dims(image_ary, axis=0)
    return image_ary
```

```

# DO NOT CHANGE THIS CELL
# configure input and output
inference_input=tritonhttpclient.InferInput(input_name, input_shape, input_dtype)
output=tritonhttpclient.InferRequestedOutput(output_name)

# create one large array by concatenating inference results on individual tiles
prediction_y=[]
for each_y in range(0, 20):
    prediction_x=[]
    for each_x in range(20):
        each_file=f'tile_{each_y}_{each_x}.png'
        if each_file in os.listdir(os.getenv('NEPAL_VV_PATH')):
            # preprocess input tiles
            image_ary=get_image_ary(each_file)
            # generate inference with Triton
            inference_input.set_data_from_numpy(image_ary)
            response=triton_client.infer(model_name,
                                         model_version=model_version,
                                         inputs=[inference_input],
                                         outputs=[output])

            logits=response.as_numpy(output_name)
            logits=np.asarray(logits, dtype=np.float32)
            mask=np.argmax(logits, axis=3)[0]
            prediction_x.append(mask)
        else:
            prediction_x.append(np.zeros((512, 512)))
    prediction_y.append(np.concatenate(prediction_x, axis=1))
prediction=np.concatenate(prediction_y, axis=0)

print(f'The result predictions has a shape of {prediction.shape}.')

```

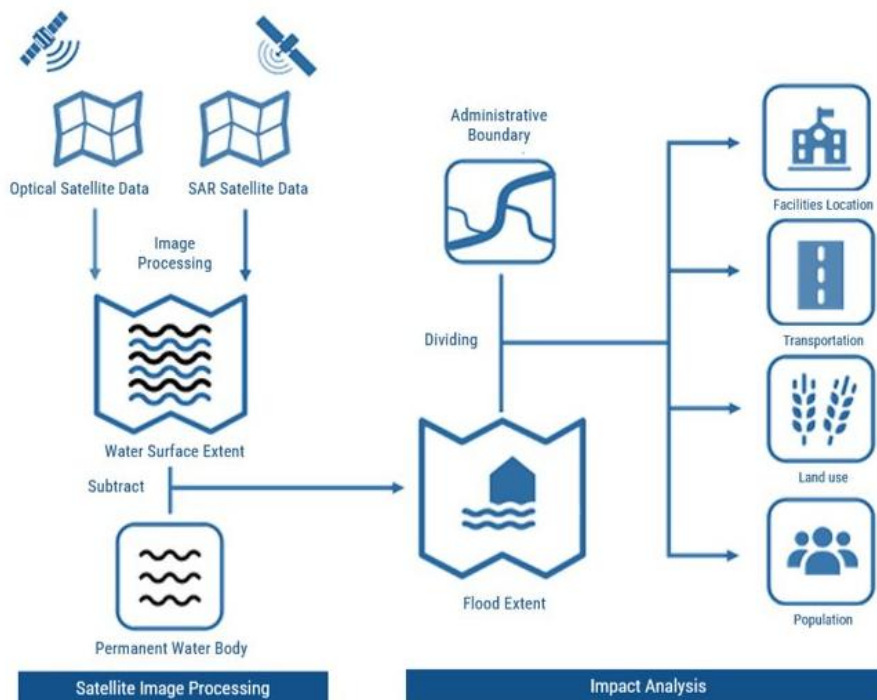
Impact Analysis

When performing an impact analysis, we are concerned with:

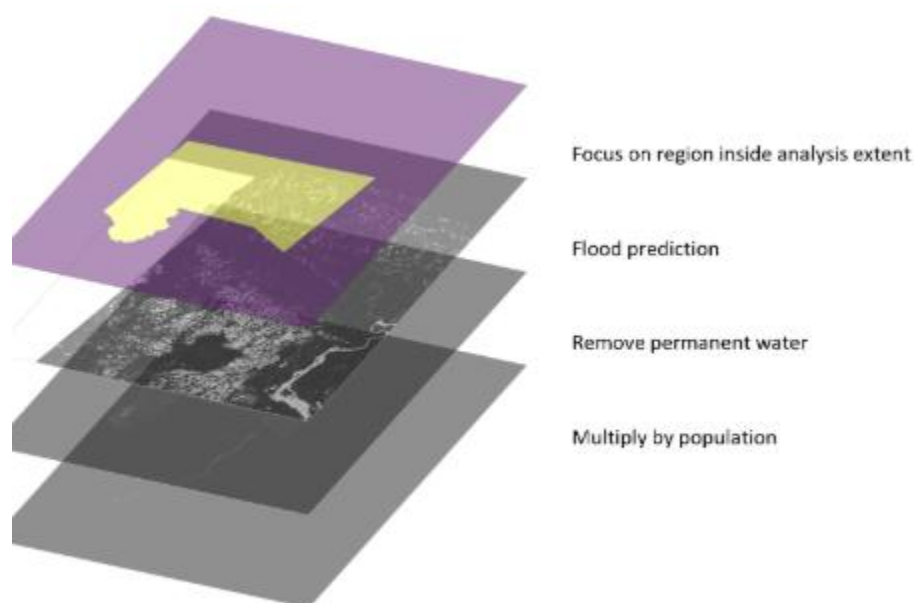
- What is the extent of the flood?
- How many people are likely affected or located within flood affected areas?
- What is the estimated number of critical facilities located within the flood affected areas?

We will use baseline data, which refers to information that provides humanitarian actors with a pre-disaster picture of the potentially affected populations, critical infrastructure, basic geography, and other issues that will become crucial when disaster strikes. Baseline data are obtained from several different sources, each responsible for ensuring accuracy and integrity. Some examples of commonly used baseline data are:

- Humanitarian Data Exchange
- Common Operational Dataset
- WorldPop
- OpenStreetMap
- Esri 2020 Land Cover Map



To estimate the population potentially affected by the flood, we will subtract the permanent water from the AI-based water extent. This will be the flood extent that didn't have water pre-disaster. We then intersect the flood extent with the open-source spatial demographic dataset [WorldPop](#). It is important to notice that there is a discrepancy between Sentinel-1 10-meter resolution and the 100-meter resolution of WorldPop. Resampling the population layer is required to align it with the flood extent before computing any impact analysis. The area of interest for the impact analysis includes the Nawalparasi West, Rupandehi and Kapilbastu districts, and Lumbini province. We curated the permanent water layer and the WorldPop dataset to easily compute the impact analysis.




```
# DO NOT CHANGE THIS CELL
# load baseline data
analysis_extent=mpimg.imread('UNOSAT_case_study/baseline_maps/analysis_extent.png')
permanent_water=mpimg.imread('UNOSAT_case_study/baseline_maps/permanent_water.png')
world_pop=mpimg.imread('UNOSAT_case_study/baseline_maps/world_pop.png')

all_maps=[{'name': 'Area of Interest', 'map': mpimg.imread('UNOSAT_case_study/baseline_maps/analysis_extent_sample.PNG')},
          {'name': 'Permanent Water Region', 'map': mpimg.imread('UNOSAT_case_study/baseline_maps/permanent_water_sample.PNG')},
          {'name': 'Population', 'map': mpimg.imread('UNOSAT_case_study/baseline_maps/world_pop_sample.PNG')}]

# plot baseline data
fig, ax=plt.subplots(1, 3, figsize=(30, 10))

for idx, map in enumerate(all_maps):
    ax[idx].imshow(map['map'])
    ax[idx].get_yaxis().set_visible(False)
    ax[idx].get_xaxis().set_visible(False)
    ax[idx].set_title(map['name'])
```

```
# DO NOT CHANGE THIS CELL
# make a copy of the water extent
flood_extent=prediction.copy()

# focus on the region inside the analysis extent only
flood_extent[analysis_extent==0]=0

# subtract permanent water
flood_extent[permanent_water!=0]=0

# calculate the potentially affected population resampling the WorldPop resolution to the Sentinel-1 resolution.
affected_population=np.sum(world_pop*255*np.float32(flood_extent)/98.5066598616484)

print(f"Population potentially affected: {affected_population}.")
```

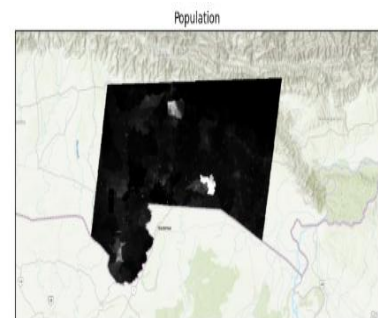
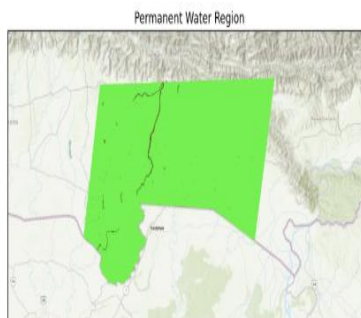
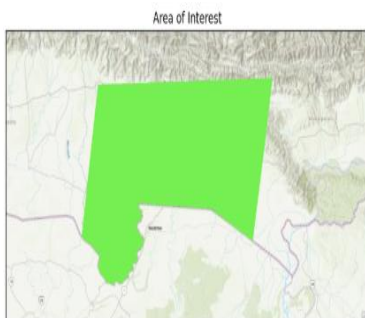
```
# DO NOT CHANGE THIS CELL
# plot layers of the impact analysis
fig=plt.figure(figsize=(10, 10))
ax=plt.subplot(111)

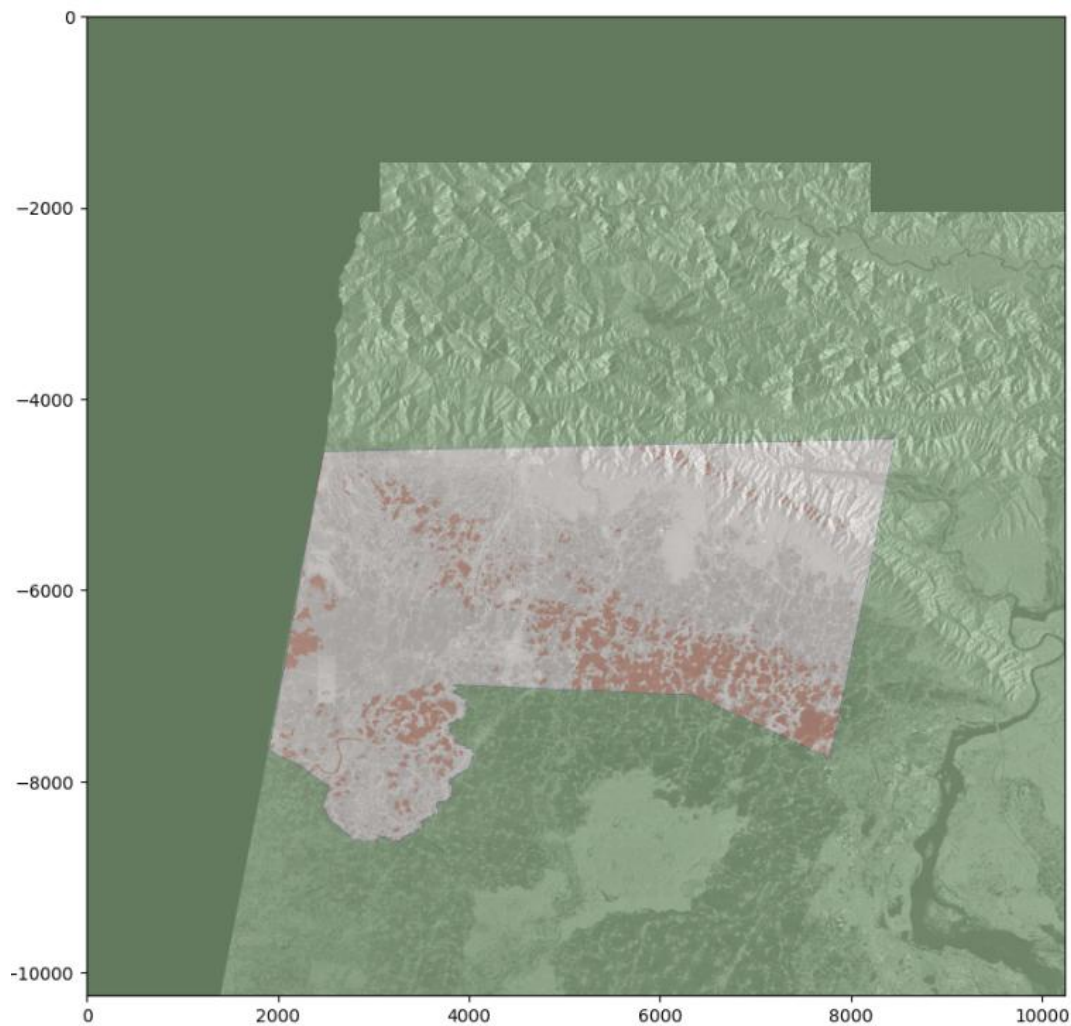
# plot map (by tiles)
for each_y in range(20):
    for each_x in range(20):
        each_file=f'tile_{each_y}_{each_x}.png'
        if each_file in os.listdir(os.environ['NEPAL_VV_PATH']):
            image=RGB_composite(each_file)
            plt.imshow(image[:, :, 1], extent=(512*each_x, 512*(each_x+1), -512*(each_y+1), -512*each_y), cmap='gray')

# plot predicted flood extent
plt.imshow(flood_extent, extent=(0, 10240, -10240, 0), cmap='Reds', alpha=0.5, vmax=3)

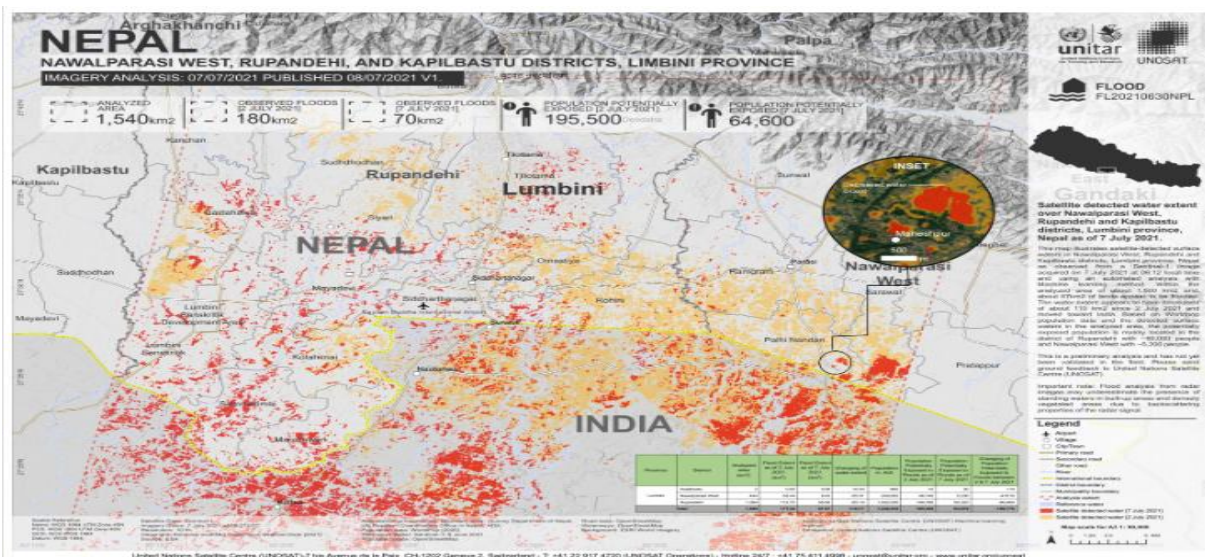
# plot area of interest
plt.imshow(analysis_extent, extent=(0, 10240, -10240, 0), cmap='ocean', alpha=0.2)

ax.set_xlim([0, 10240])
ax.set_ylim([-10240, 0])
ax.set_facecolor("black")
plt.show()
```





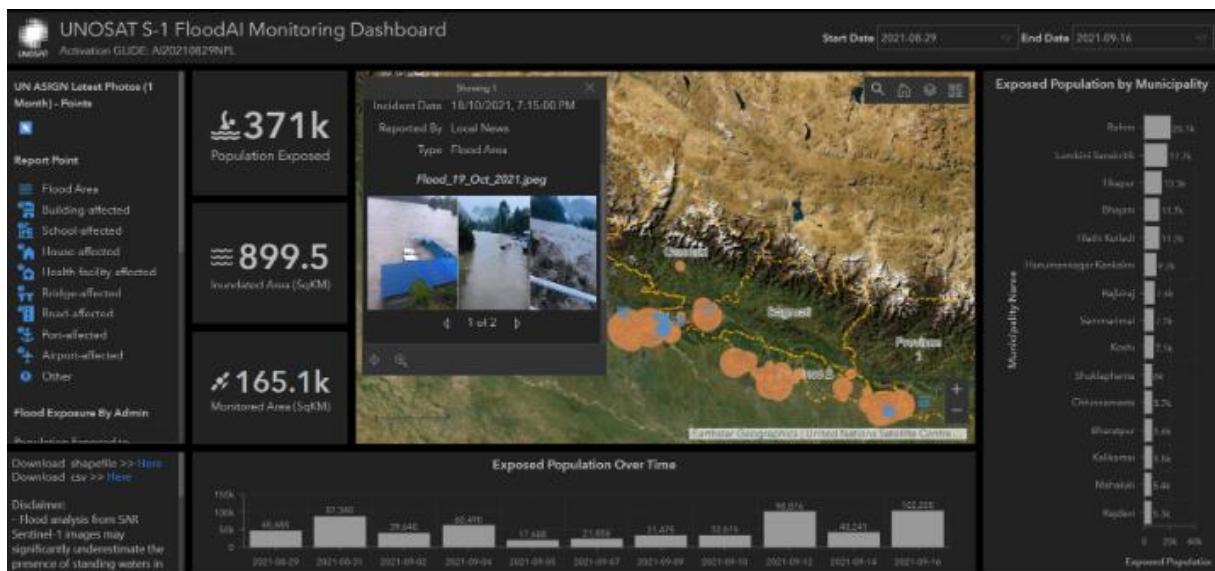
The United Nations Satellite Centre (UNOSAT) map, displayed below, indicated a potential expose population exposed of 195,000 on June 2nd. Is it in line with the number you just obtained.



Near Real-Time Flood Dashboard

To run flood detection at scale and respond to a growing need for daily updates, it is crucial to develop end-to-end pipelines where satellite images are automatically downloaded and processed by machine learning algorithms to output flood extent. Tools that provide access and disseminate this information could play a key role, such as the [operational dashboards](#) that UNOSAT developed for the UN Regional Coordination Office in Nepal.

To ensure timely and accurate updates, the AI-generated flood extents are reviewed and corrected as necessary by the UNOSAT Rapid Mapping team, before feeding the dashboard for end user consumption. Another tailor-made feature was the integration of [UN ASIGN](#), an application that was shared with field officers across the country to upload geotagged information and photos in real time. The UN ASIGN application provided a way to collect and centralize ground data, which was extremely useful during the national lockdown. The visualization of all the data on the same platform provides a comprehensive overview of the area of interest despite movement restrictions.



Conclusion: From Satellite Signal to Life-Saving Insight

The journey we have completed, from raw **Sentinel-1 satellite data** to a deployed, real-time flood detection model, represents the full, end-to-end lifecycle of a mission-critical AI system. This is more than just an academic exercise; it is a blueprint for transforming **Computer Vision** models into tools of immediate global impact.

We began by mastering the challenge of scale, employing hardware-accelerated techniques like **DALI (NVIDIA Data Loading Library)** to efficiently process and augment massive geospatial

datasets. This ability to handle high-throughput data is the foundation of any system designed for global coverage and rapid response.

Next, we transitioned to the core of the solution, leveraging **Transfer Learning** within the **NVIDIA TAO Toolkit** to train a high-accuracy **U-Net semantic segmentation model**. This step highlighted how pre-trained models, when efficiently fine-tuned, drastically reduce development time while delivering enterprise-grade performance.

Finally, we crossed the critical chasm from training to deployment. By implementing the model on the **NVIDIA Triton Inference Server** and converting it into a **TensorRT engine**, we optimized inference performance for both high-throughput and low-latency environments. This optimization, demonstrated by the throughput vs. latency analysis, ensures that the resulting flood maps are generated not just accurately, but in real-time—a necessity for effective emergency management platforms like those used by organizations such as UNOSAT.

The resulting flood event case study illustrated the true purpose of this entire pipeline: **actionable intelligence**. By integrating this technology, human response teams can shift from reactive information gathering to proactive, targeted aid distribution. This technology is a force multiplier, giving humanitarian efforts the speed and precision needed to mitigate the devastating effects of natural disasters.

The techniques covered here—accelerated pre-processing, efficient training via transfer learning, and high-performance deployment—are foundational. They can be generalized far beyond flood detection to address wildfires, deforestation, agricultural monitoring, and other critical global challenges. You have now acquired the practical skills to build and deploy intelligent, impactful systems that harness the power of AI for a better, safer world.

References

This section provides the original sources, technical documentation, and foundational research papers for the concepts, algorithms, and tools used throughout this book to build the end-to-end flood detection system.

I. Foundational Deep Learning Architectures

The core segmentation task is built upon established deep learning models, which must be formally acknowledged.

1. **U-Net Architecture (Semantic Segmentation Model)**
 - Ronneberger, O., Fischer, P., & Brox, T. (2015). **U-Net: Convolutional Networks for Biomedical Image Segmentation**. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. Springer, Cham.
 - *Source*: The paper that introduced the foundational encoder-decoder architecture used for high-accuracy pixel-level classification.
2. **ResNet (Backbone for U-Net)**
 - He, K., Zhang, X., Ren, S., & Sun, J. (2016). **Deep Residual Learning for Image Recognition**. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770-778.
 - *Source*: The seminal work on residual blocks, which provides the highly efficient and effective feature extraction backbone for the U-Net model used in this system.

II. Data Sources and Acquisition

The system relies entirely on publicly available, mission-critical satellite data.

3. **Sentinel-1 Satellite Mission**
 - European Space Agency (ESA) & Copernicus Programme. **Sentinel-1 User Handbook**.
<https://documentation.dataspace.copernicus.eu/Data/SentinelMissions/Sentinel1.html>
 - *Source*: The official source of the Synthetic Aperture Radar (SAR) data, which is the primary input for the flood detection model.

III. Accelerated Development and Deployment Tools

This section credits the software frameworks used to achieve high performance in training and inference.

4. **NVIDIA DALI (Data Loading Library)**
 - NVIDIA Corporation. **NVIDIA DALI Documentation**.
<https://docs.nvidia.com/deeplearning/dali/user-guide/docs/index.html>

- *Source:* The official guide for the hardware-accelerated, GPU-native data pre-processing and augmentation library, crucial for overcoming I/O bottlenecks.
- 5. **NVIDIA TAO Toolkit (Transfer Learning Framework)**
 - NVIDIA Corporation. **NVIDIA TAO Toolkit Documentation.** <https://docs.nvidia.com/tao/tao-toolkit/index.html>
 - *Source:* Documentation for the framework that simplifies the application of Transfer Learning, pruning, and model export to TensorRT.
- 6. **NVIDIA TensorRT (High-Performance Inference)**
 - NVIDIA Corporation. **NVIDIA TensorRT Developer Guide.** <https://docs.nvidia.com/deeplearning/tensorrt/latest/index.html>
 - *Source:* The guide for the platform that optimizes trained models into highly efficient inference engines for deployment.
- 7. **NVIDIA Triton Inference Server**
 - NVIDIA Corporation. **NVIDIA Triton Inference Server Documentation.** <https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/index.html>
 - *Source:* The platform used to deploy the TensorRT-optimized model, managing concurrent execution, batching, and enabling real-time network API access.

IV. Real-World Application and Context

This entry grounds the technical solution in its real-world humanitarian context.

- 8. **UNOSAT (United Nations Satellite Centre)**
 - United Nations Institute for Training and Research (UNITAR) - UNOSAT. **Official Website and Disaster Mapping Reports.** <https://unosat.org/>
 - *Source:* The organization and the context for the flood event case study, illustrating how the model's output is translated into humanitarian and emergency response action.